OBJECT-ORIENTED ANALYSIS AND DESIGN PRINCIPLES AND APPLICATIONS

DR. DEEPANJAL SHRESTHA

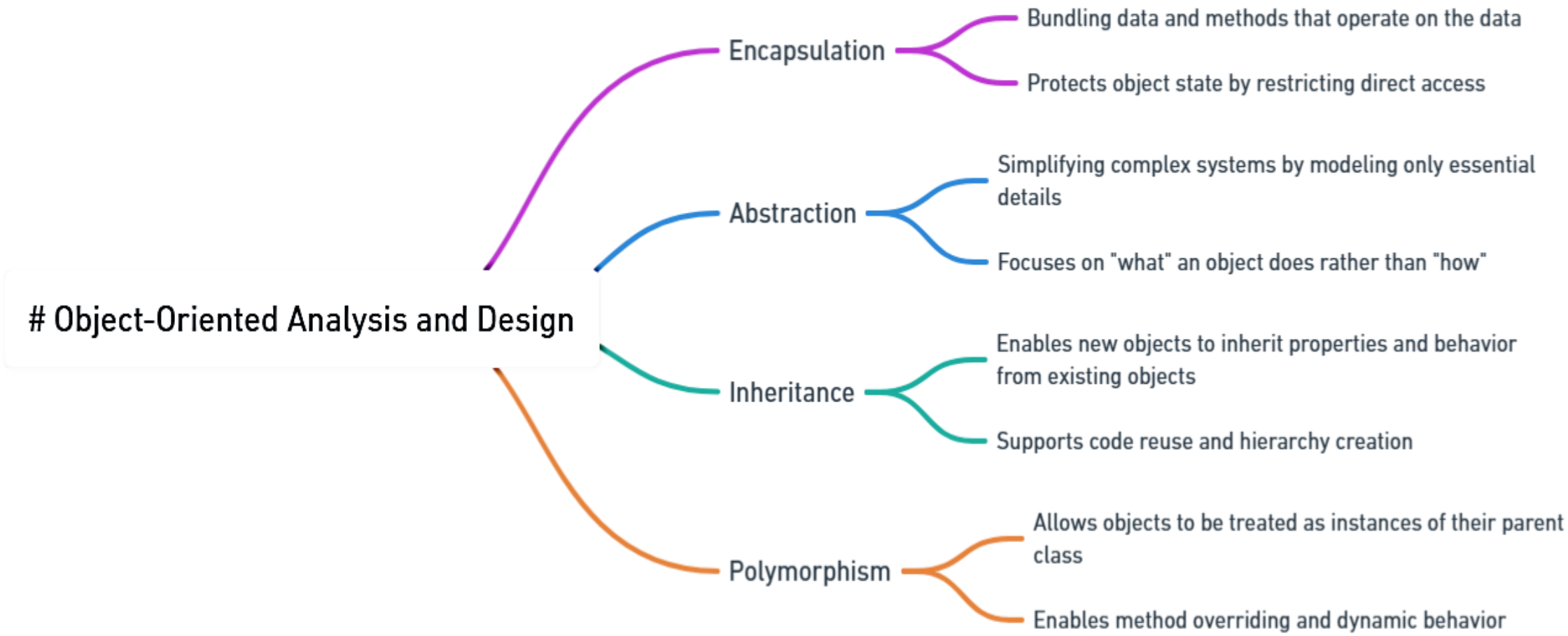
ASSOCIATE PROFESSOR (COMPUTER SCIENCE)

SCHOOL OF ENGINEERING

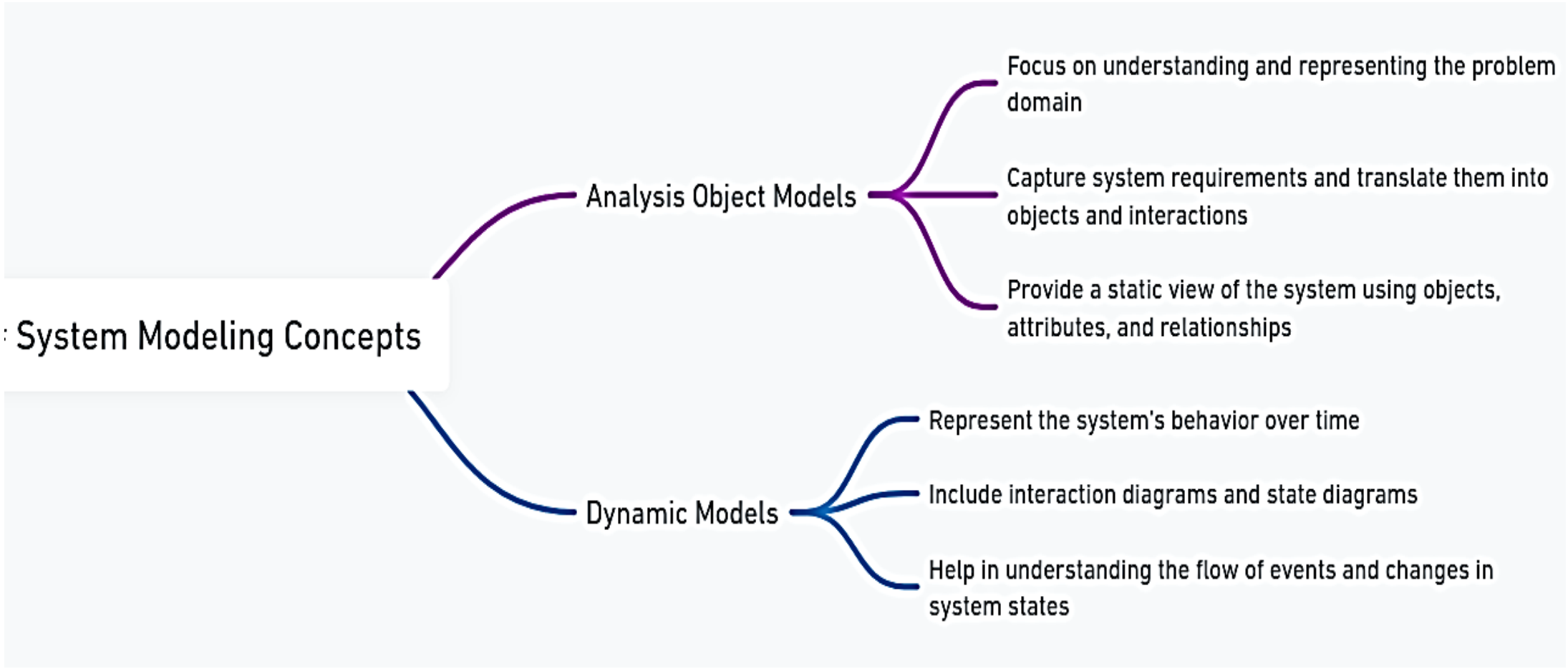
OOAD PRINCIPLES

- Encapsulation
  - Bundling data and methods that operate on the data.
  - Protects object state by restricting direct access.
- Abstraction
  - Simplifying complex systems by modeling only essential details.
  - Focuses on "what" an object does rather than "how."
- Inheritance
  - Enables new objects to inherit properties and behavior from existing objects.
  - Supports code reuse and hierarchy creation.
- Polymorphism
  - Allows objects to be treated as instances of their parent class.
  - Enables method overriding and dynamic behavior.

OOAD PRINCIPLES



SYSTEM MODELING CONCEPTS



ANALYSIS OBJECT MODELS

- Class Diagram:
  - Represents the static structure of the system.
  - Shows classes, attributes, methods, and relationships (e.g., inheritance, association).
  - Example: A Student Management System class diagram with classes like Student, Course, Instructor, and their relationships.
- Object Diagram:
  - A snapshot of the system at a specific point in time.
  - Example: A Shopping Cart object diagram with objects like Cart, Item, User, and their interconnections.
- Domain Model:
  - Captures the key concepts and their relationships in the problem domain.
  - Example: For an E-commerce System, the domain model may include Product, Order, Customer, and Payment.

DYNAMIC MODELS

- Sequence Diagram:
  - Models the interactions between objects in a specific scenario.
- Example: For a Login System, a sequence diagram may show interactions like:
  - User → System: Enter credentials.
  - System → Database: Verify credentials.
  - Database → System: Return status.
  - System → User: Success or error message.

DYNAMIC MODELS

- State Diagram:
  - Describes the states an object "**can be in**" and transitions based on events.
- Example: A Ticket Booking System:
  - States: Available, Booked, Paid, Cancelled.
  - Transitions: Book → Paid → Cancelled.
- Activity Diagram:
  - Represents workflows of actions or operations.
- Example: Order Fulfillment Process:
  - States: Receive Order, Process Payment, Pack Items, Ship.

ENTITY, BOUNDARY, AND CONTROL OBJECTS

- **Entity Objects**
  - Represent core data and business logic.
  - Persist in the system as part of the database or internal storage.
  - Example: A "Customer" object with attributes like name, address, and purchase history.
- **Boundary Objects**
  - Act as interfaces between the system and external entities (users, devices, or other systems).
  - Ensure a smooth flow of information to and from the system.
  - Example: A "Login Screen" object that captures user credentials.

CONTROL OBJECTS

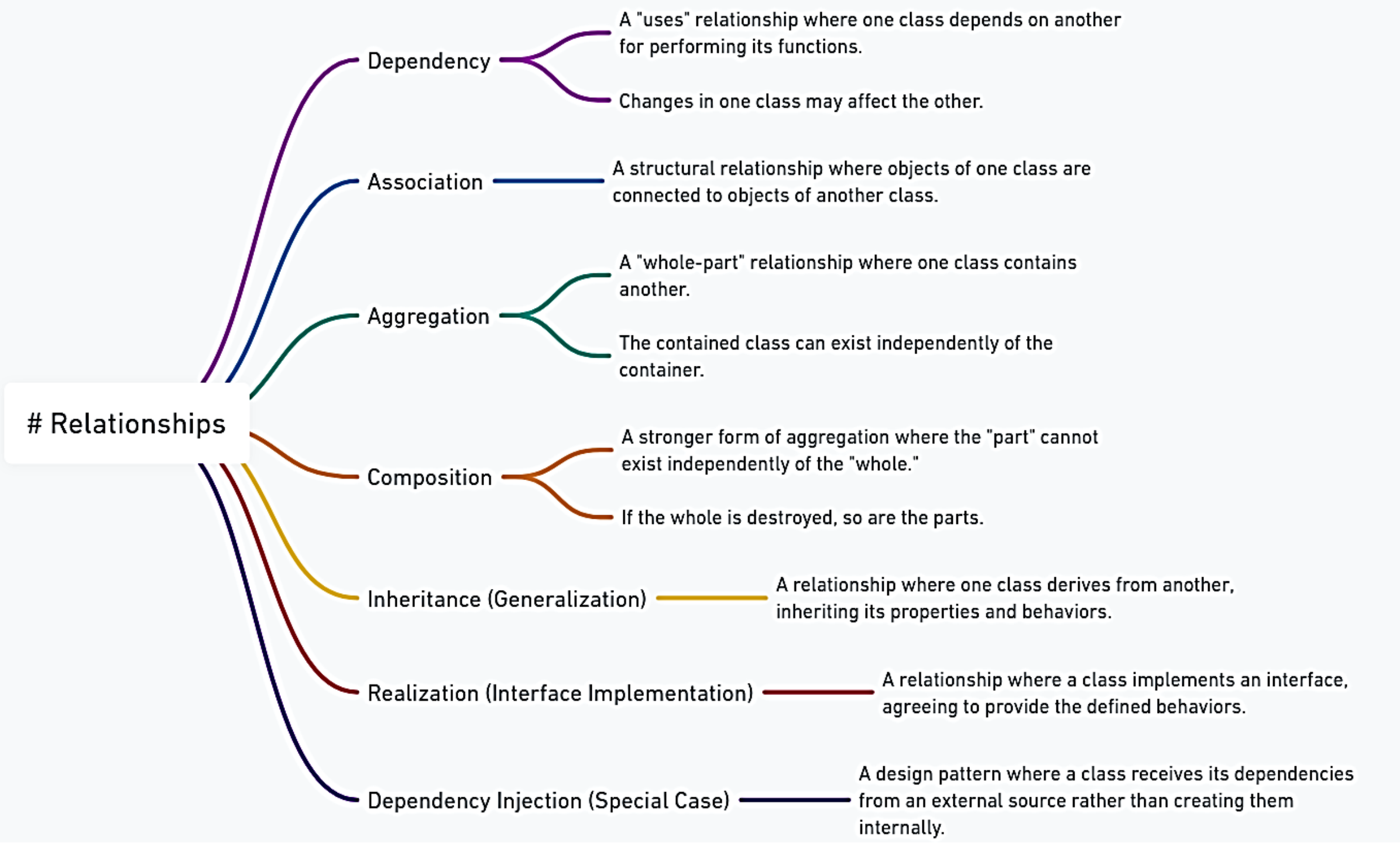
- **Control Objects**
- Coordinate and manage the flow of control among other objects.
- Implement use case logic by connecting entity and boundary objects.
- Example: An "Order Processing" object that validates a customer's order and initiates payment processing.

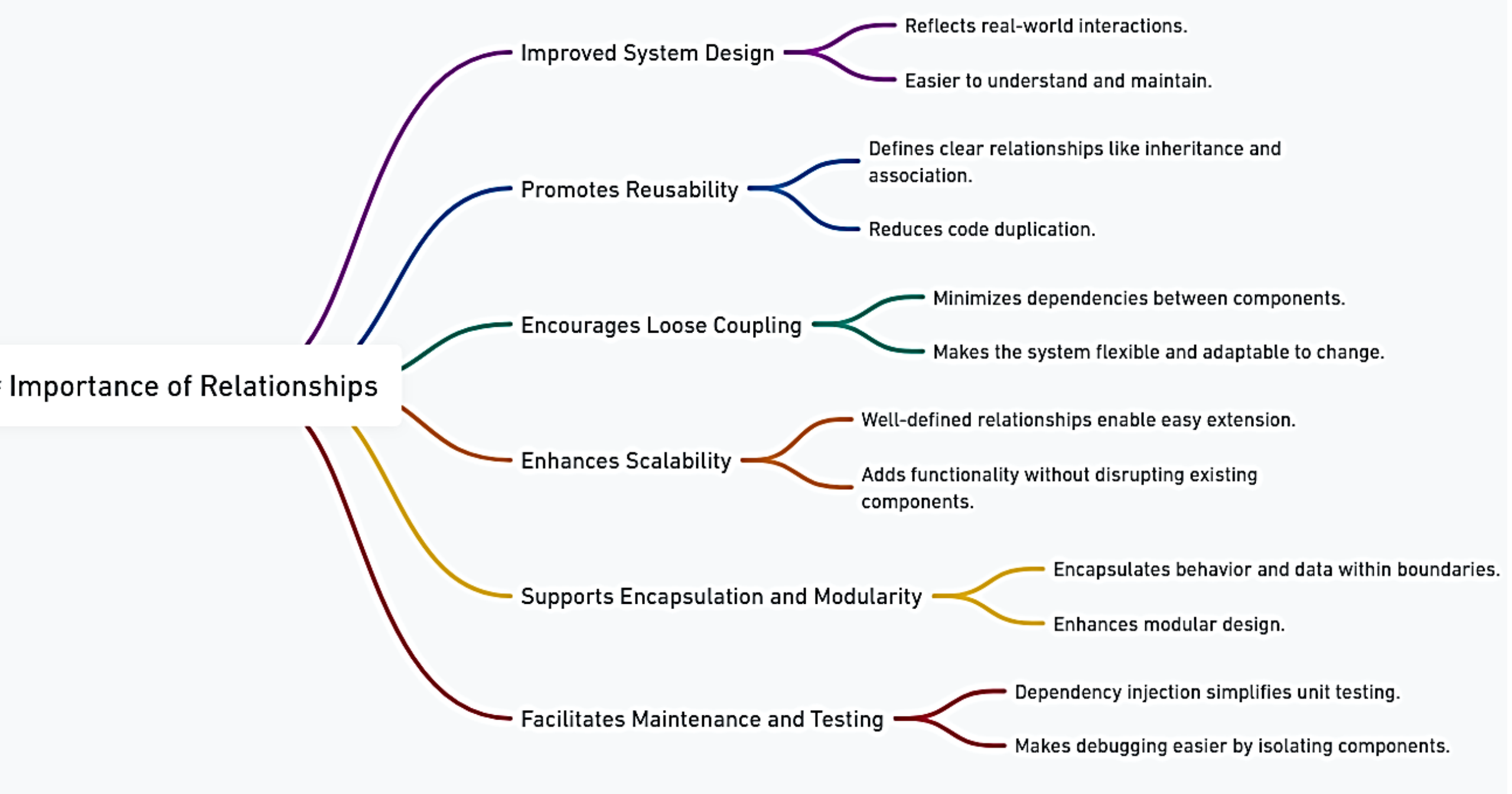
RELATIONSHIP IN OOAD

- In Object-Oriented Analysis and Design (OOAD), relationships define how different objects or classes interact and communicate with each other to form a cohesive system.
- Understanding these relationships is crucial for creating well-structured, maintainable, and scalable systems

Types of Relationship

- Dependency
- Association
- Aggregation
- Composition
- Inheritance (Generalization)
- Realization (Interface Implementation)
- Dependency Injection (Special Case)

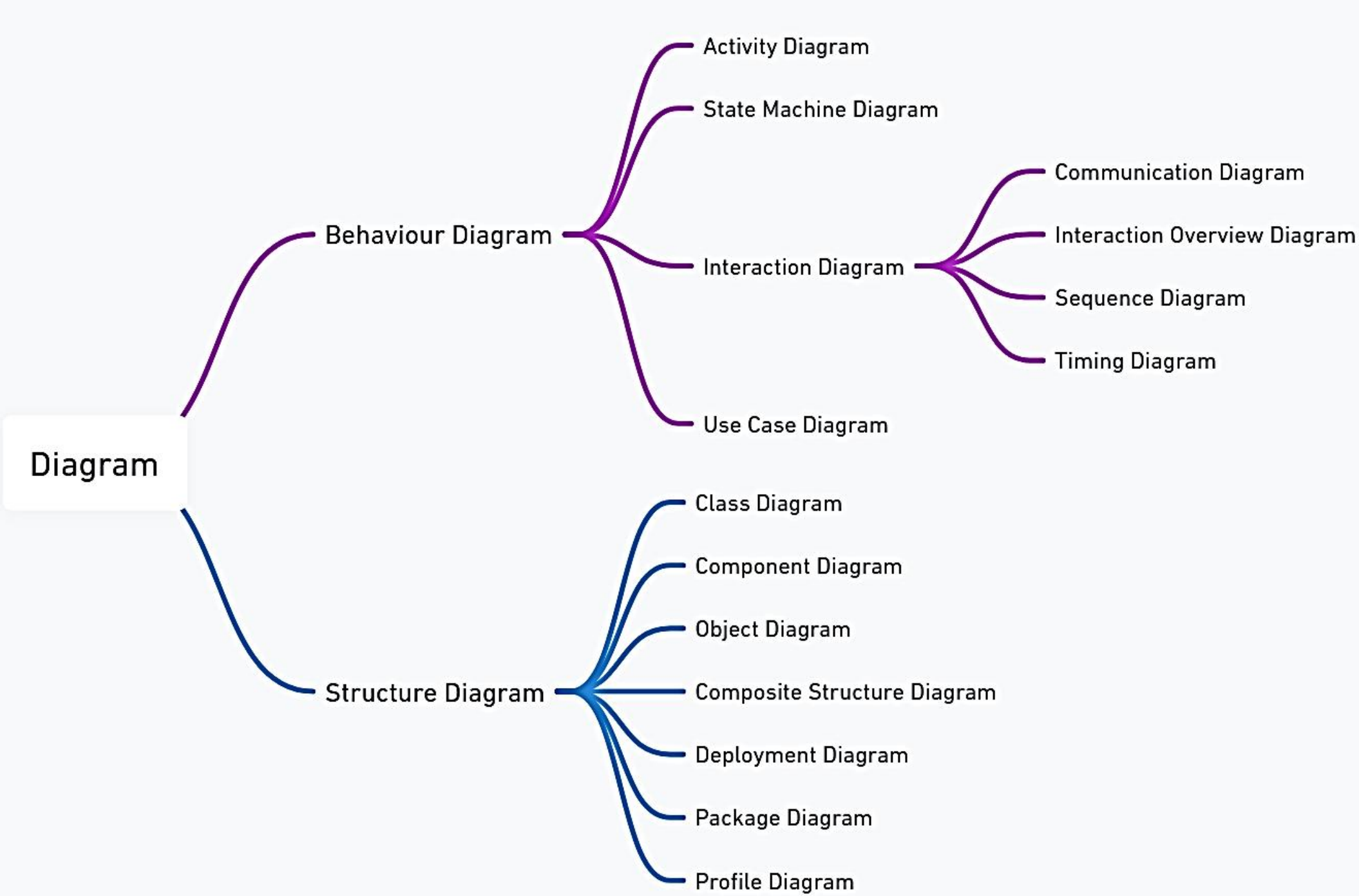

Relationships
Dependency
A "uses" relationship where one class depends on another for performing its functions.
Changes in one class may affect the other.
Association
A structural relationship where objects of one class are connected to objects of another class.
Aggregation
A "whole-part" relationship where one class contains another.
The contained class can exist independently of the container.
Composition
A stronger form of aggregation where the "part" cannot exist independently of the "whole."
If the whole is destroyed, so are the parts.
Inheritance (Generalization)
A relationship where one class derives from another, inheriting its properties and behaviors.
Realization (Interface Implementation)
A relationship where a class implements an interface, agreeing to provide the defined behaviors.
Dependency Injection (Special Case)
A design pattern where a class receives its dependencies from an external source rather than creating them internally.


Importance of Relationships
Improved System Design
Reflects real-world interactions.
Easier to understand and maintain.
Promotes Reusability
Defines clear relationships like inheritance and association.
Reduces code duplication.
Encourages Loose Coupling
Minimizes dependencies between components.
Makes the system flexible and adaptable to change.
Enhances Scalability
Well-defined relationships enable easy extension.
Adds functionality without disrupting existing components.
Supports Encapsulation and Modularity
Encapsulates behavior and data within boundaries.
Enhances modular design.
Facilitates Maintenance and Testing
Dependency injection simplifies unit testing.
Makes debugging easier by isolating components.

THE UNIFIED MODELING LANGUAGE (UML)

- UML or Unified Modeling Language comes from **Rumbaugh, Booch, and Jacobson** (the three amigos) who combined efforts to standardize on one modeling language
- This is primarily a graphical communication mechanism for developers and customers
- We will learn some, but not all, of the UML **(*)**
  - It is very complex, few understand all of it

UML DIAGRAMS



ROLE OF UML DIAGRAMS

- Use case diagrams (Fulfill functionality)
- Class diagrams(Depict static view and relationship)
- Object Diagrams(Depict view of instance of class)
- Sequence Diagrams(Depict message over time line)
- Collaboration Diagrams (Like sequence diagram but define object roles)
- State chart Diagrams(Depicts states of the object)
- Activity Diagrams(Dynamic view like flow chart)
- Component Diagrams(Depict on the components)
- Deployment Diagrams(Depict physical layout of the system)

USE CASE DIAGRAMS

- Use cases describe how users interact with a system to achieve specific goals.
- They focus on the functional requirements of a system.
- **Purpose**:
  - Identify interactions between users and the system.
  - Define high-level requirements.
- **Key Components**:
  1. **Actors**: Users or systems interacting with the system.
  2. **Use Cases**: Specific functionalities or processes.
  3. **Relationships**: Dependencies between actors and use cases.

KEY COMPONENTS AND BENEFITS OF USE CASES

- **Actor:** The user or entity interacting with the system (e.g., customer, admin).
- **Goal:** What the actor wants to achieve (e.g., make a payment).
- **System:** The software performing the tasks.
- **Steps:** Actions taken by the actor and system to achieve the goal
- Benefits
  - Clarifies system functionality.
  - Helps in identifying missing requirements.
  - Guides design and development.
  - Serves as a basis for testing.

Key Rule:
Actors, whether external or internal, always represent **something that performs actions or interacts with the system**, but they do not belong to the system itself in the context of the use case being modeled. An actor's role is to initiate or participate in a use case by exchanging information with the system.

AN EXAMPLE OF AN ACTOR

- In **"IssueTicket"** use case, is the **printing system an actor?**
- **Yes,** it can be considered an actor as the printing system plays an external role in the ticket issuance process, it qualifies as a secondary actor
- It supports the primary goal of the passenger (the primary actor) to obtain a ticket.
- It communicates with the metro ticketing system to fulfill a specific task (printing the ticket).

Even though the printing system might be physically part of the metro ticketing infrastructure, it acts externally in the context of this specific use case because it operates as a separate system that the ticketing system must interact with.

APPLICATION AND TYPES OF USE CASES

- **High-Level Use Cases:** Broad descriptions (e.g., "Place an order").
- **Detailed Use Cases:** Step-by-step interactions (e.g., "Add item to cart, enter payment details, confirm order").
- **Applications:**
  - **Requirement Analysis:** Understanding what the system must do.
  - **System Design:** Structuring the software to meet goals.
  - **Testing:** Creating test cases to ensure functionality.
  - **Documentation:** Providing stakeholders with clear insights into the system.

USE CASE: SUBMIT A LEAVE

Use case name	SubmitLeaveRequest
Participating actors	Initiated by Employee Communicates with Manager
Flow of events	1. The **Employee** logs into the HR portal and selects the "Submit Leave Request" option. 2. The system displays a leave request form to the **Employee**. 3. The **Employee** fills out the form by specifying the type of leave, start date, end date, and reason for the leave. Once the form is completed, the **Employee** submits the request. 4. The system records the leave request and notifies the **Manager**. 5. The **Manager** reviews the leave request and either approves or rejects it. The **Manager** may add comments if needed. 6. The system notifies the **Employee** of the **Manager's** decision, along with any comments provided.
Entry condition	- The **Employee** is logged into the HR portal.
Exit condition	- The **Employee** receives a notification indicating the leave request has been approved or rejected. - The leave request is recorded in the system with the **Manager's** decision.
Quality requirements	- The **Manager** should receive the leave request immediately after submission. - The **Employee** should receive the decision within one business day.

BRAIN TEASER

- Write a Use case for ResetPassword !!

SOLUTION

Use case name	**ResetPassword**
Participating actors	Initiated by User Communicates with System
Flow of events	1. The User selects the "Forgot Password" option on the login screen. 2. The system prompts the User to enter their registered email address. 3. The User enters the email address and submits the form. 4. The system sends a password reset link to the provided email address. 5. The User clicks on the reset link, which directs them to a secure reset password page. 6. The User enters a new password and confirms it. 7. The system validates the password and updates it in the database 8. The system notifies the User that the password reset was successful.
Entry condition	The User has a registered email address in the system.
Exit condition	The User's password is successfully updated. The User is notified of the successful reset.
Quality requirements	The reset password email should be sent within 1 minute of the request. The password reset process must be completed within 15 minutes of receiving the email.

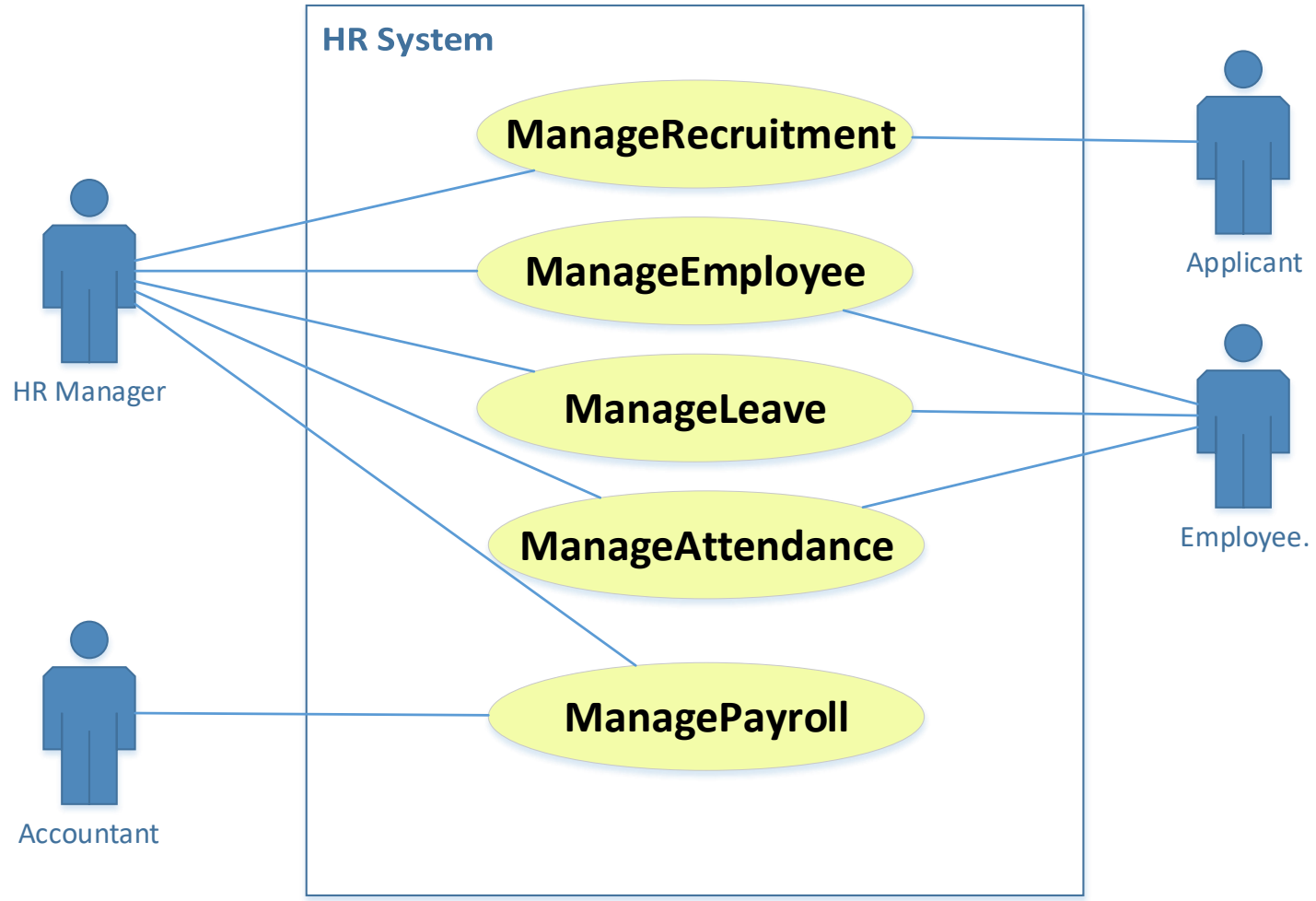
USE CASE DIAGRAM

- A use case diagram is a visual representation in UML (Unified Modeling Language) that illustrates the interactions between actors and the system through use cases. It provides a high-level overview of the functional requirements of a system.
- **Components of a Use Case Diagram**
- **Actors:** Represent the entities that interact with the system
- **Use Cases:** Represent the specific functionalities or actions that the system performs in response to an actor's interaction.
  - Example: Place Order, Login, Search Product.
- **System Boundary**: Represents the scope of the system. It defines what is included within the system and what is external to it.
  - Example: A rectangle labeled E-Commerce System containing use cases like Browse Catalog and Checkout.
- **Relationships:** Indicate how the various elements of the diagram are connected.
  - Common relationships include (Association, Include, Extend, Generalization)
- **Notes:** Provide additional information or annotations to clarify specific elements of the diagram.
  - Notation: Represented as a rectangle with a folded corner connected by a dashed line.
  - Example: A note explaining a specific constraint on a use case.

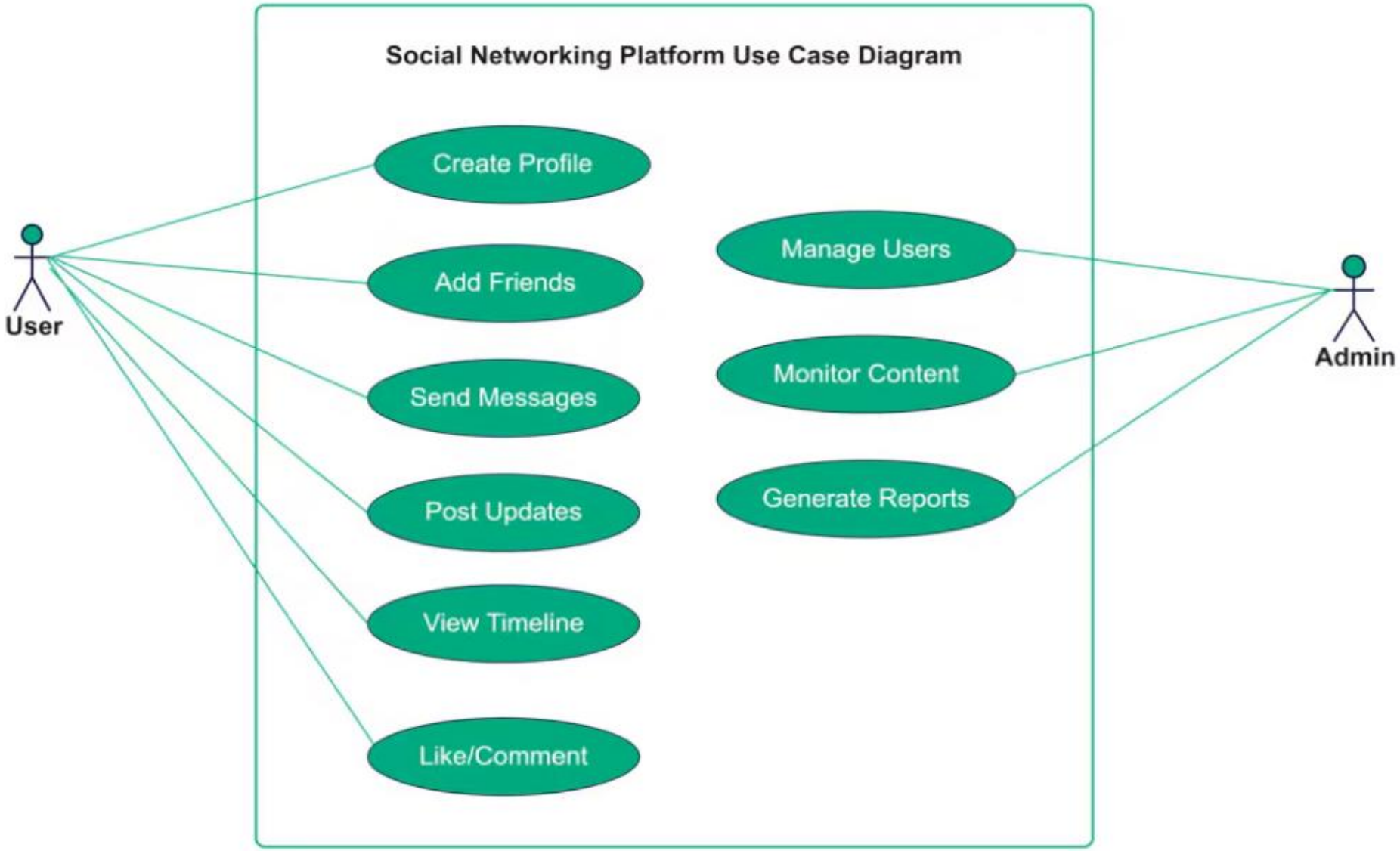
USE CASE RELATIONSHIP

Relationship	Description	Notation	Example
Association	Link between an actor and a use case	Solid line	A Passenger is associated with the "Buy Ticket" use case.
Include	Mandatory inclusion of another use case	Dashed arrow labeled «include»	The "Recharge Card" use case includes "Process Payment".
Extend	Optional extension of another use case	Dashed arrow labeled «extend»	The "Buy Ticket" use case is extended by "Print Receipt".
Generalization	Inheritance relationship between actors or use cases	Solid line with hollow arrowhead	A VIP Passenger actor inherits from the Passenger actor.

USE CASE DIAGRAM: HR SYSTEM



EXAMPLE 2: SOCIAL MEDIA



IMPORTANCE OF USE CASE DIAGRAM COMPONENTS

- **Actors:** Help identify who or what interacts with the system and their roles.
- **Use Cases:** Define the functional requirements and describe the expected behavior of the system.
- **System Boundary**: Clarifies the scope of the system and avoids scope creep.
- **Relationships**: Establish connections and dependencies between various elements, improving understanding of the system's behavior.
- **Notes:** Enhance clarity and provide context for better interpretation.

CLASS DIAGRAMS

- A **class diagram** is a type of static structure diagram in the Unified Modeling Language (UML) that describes the structure of a system. It visually represents:
- **Classes**: The building blocks of an object-oriented system.
- **Attributes**: Properties or data fields within a class.
- **Methods**: Operations or functions a class can perform.
- **Relationships**: Connections between classes, including inheritance, associations, aggregations, compositions, and dependencies.

CLASSES : KEY CONCEPTS

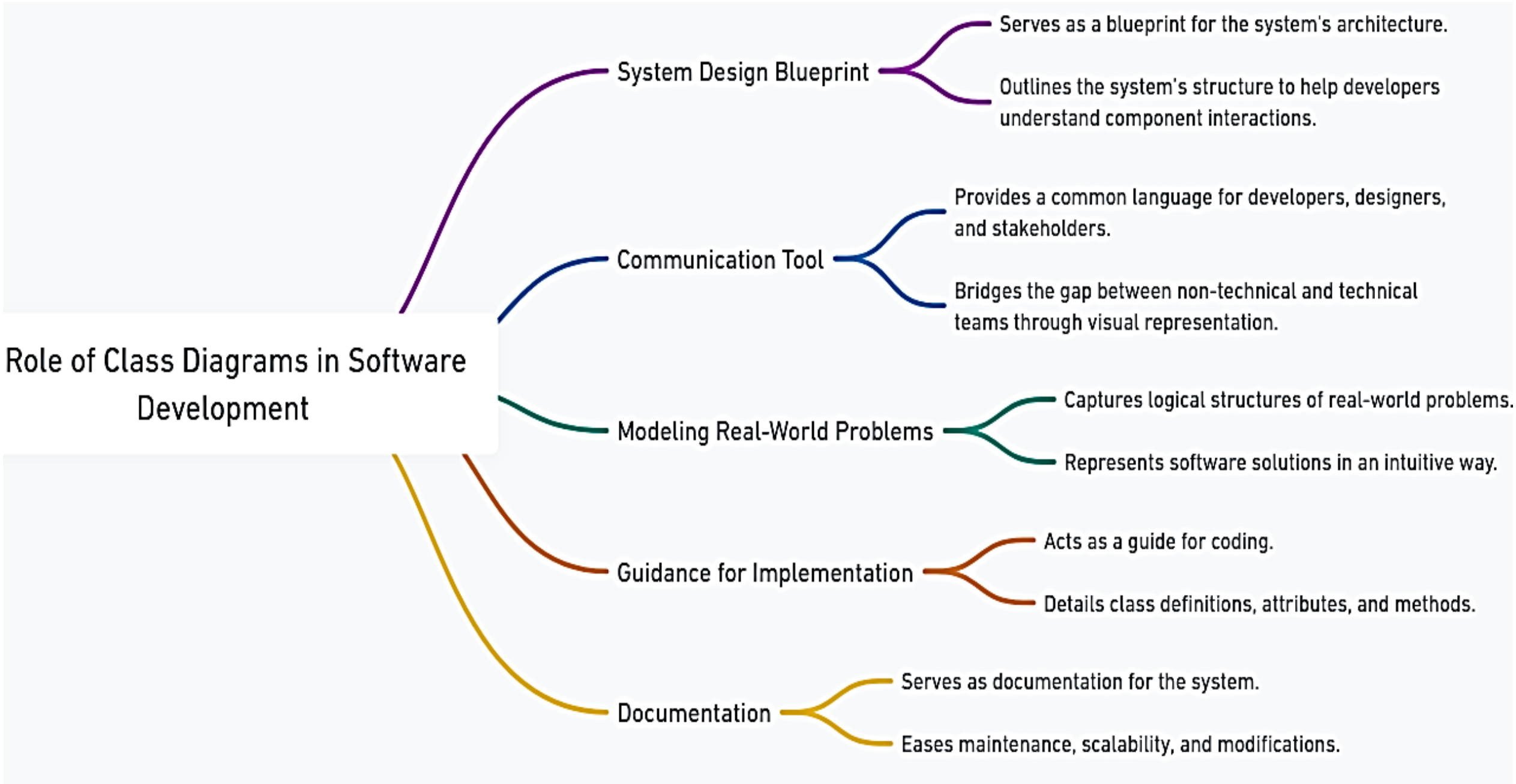
Key Concepts

Classes:

- Blueprint for objects.
- Composed of:
  - Attributes: Define properties (e.g., name, age).
  - Methods: Define behavior (e.g., calculateSalary()).

Objects:

- Instances of classes.
- Represent real-world entities.


Role of Class Diagrams in Software Development
System Design Blueprint
Serves as a blueprint for the system's architecture.
Outlines the system's structure to help developers understand component interactions.
Communication Tool
Provides a common language for developers, designers, and stakeholders.
Bridges the gap between non-technical and technical teams through visual representation.
Modeling Real-World Problems
Captures logical structures of real-world problems.
Represents software solutions in an intuitive way.
Guidance for Implementation
Acts as a guide for coding.
Details class definitions, attributes, and methods.
Documentation
Serves as documentation for the system.
Eases maintenance, scalability, and modifications.

RESPONSIBILITIES OF CLASS DIAGRAMS

1. **Defining Structure**
    1. Describes the static structure of the system, including classes, attributes, and methods.
2. **Establishing Relationships**
    1. Models relationships between classes like inheritance, associations, and dependencies, clarifying object interactions.
3. **Facilitating Reusability**
    1. Helps identify reusable components and promotes modular design.
4. **Supporting Object-Oriented Principles**
    1. Enforces principles like encapsulation, inheritance, and polymorphism by showing how classes interact and share functionality.
5. **Identifying System Behavior**
    1. Provides insight into the system's behavior by detailing how classes interact through their methods and relationships.
6. **Ensuring Consistency**
    1. Maintains consistency between the design and implementation phases by offering a detailed visual plan.

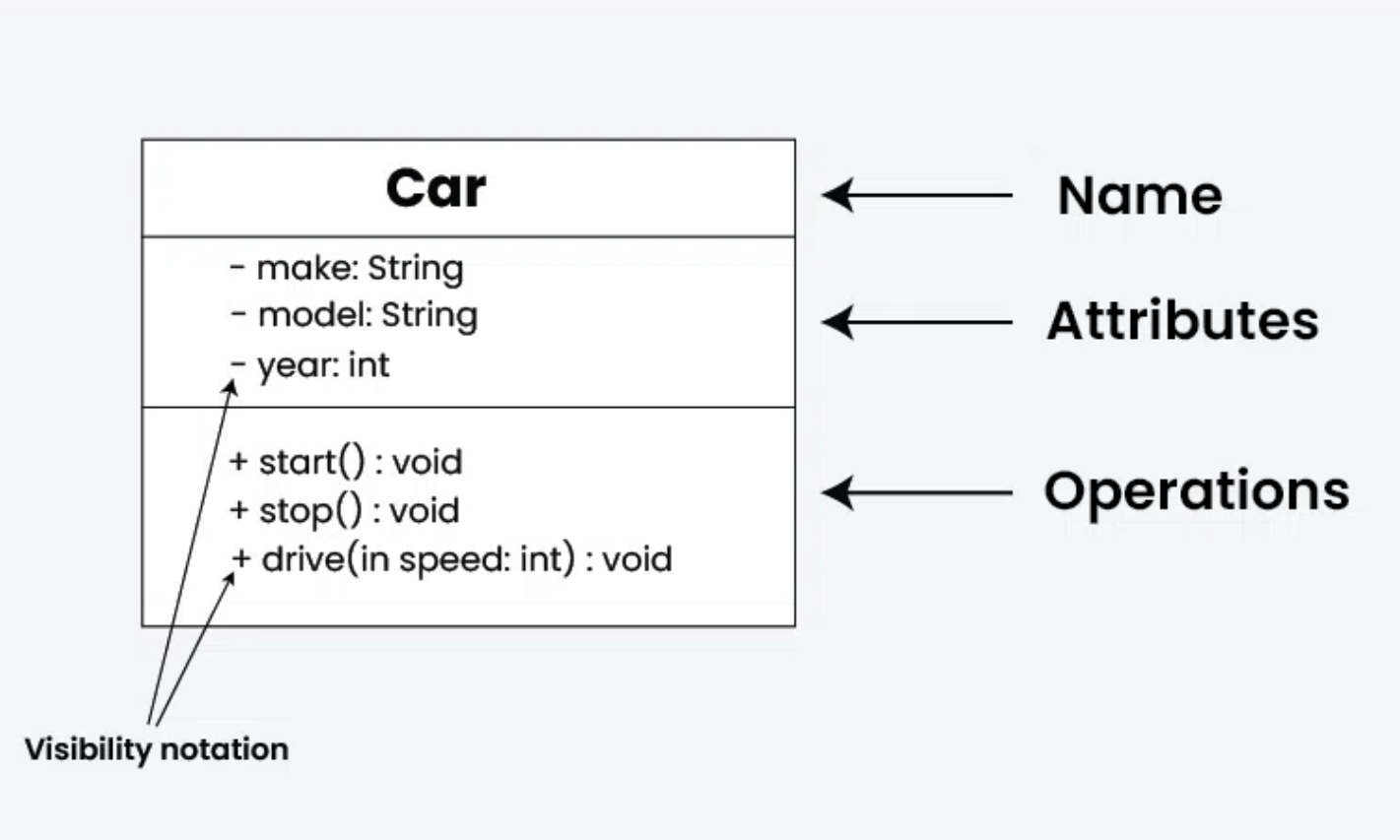
USES IN SOFTWARE DEVELOPMENT

- **Static View**: Represents the structure of a system without showing dynamic behavior.
- **System Blueprint**: Acts as a detailed plan for developers.
- **Code Generation**: Many tools can generate code skeletons directly from class diagrams.
- **Documentation**: Provides clear documentation of a system's structure.

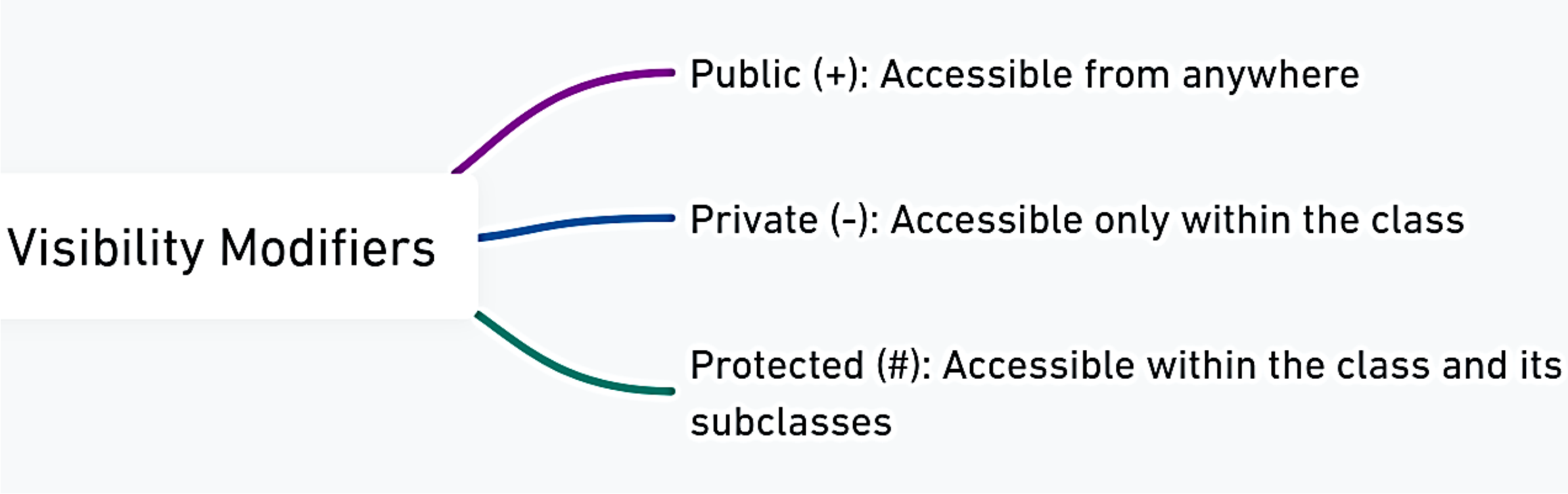
ADVANTAGES

- Clarifies system structure.
- Simplifies communication among developers.
- Helps identify design issues early.
- Facilitates object-oriented design.

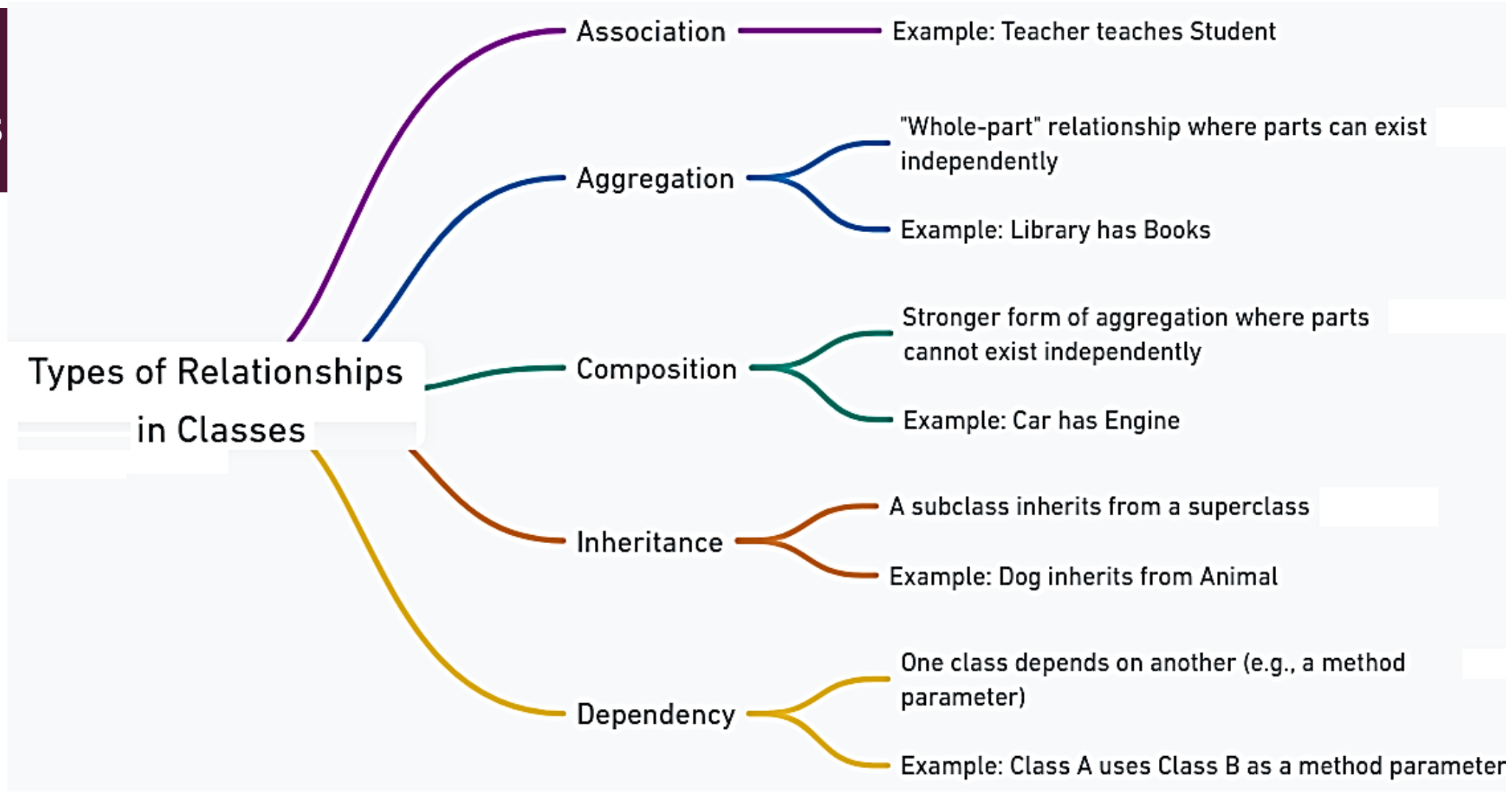
CLASS DIAGRAM NOTATION



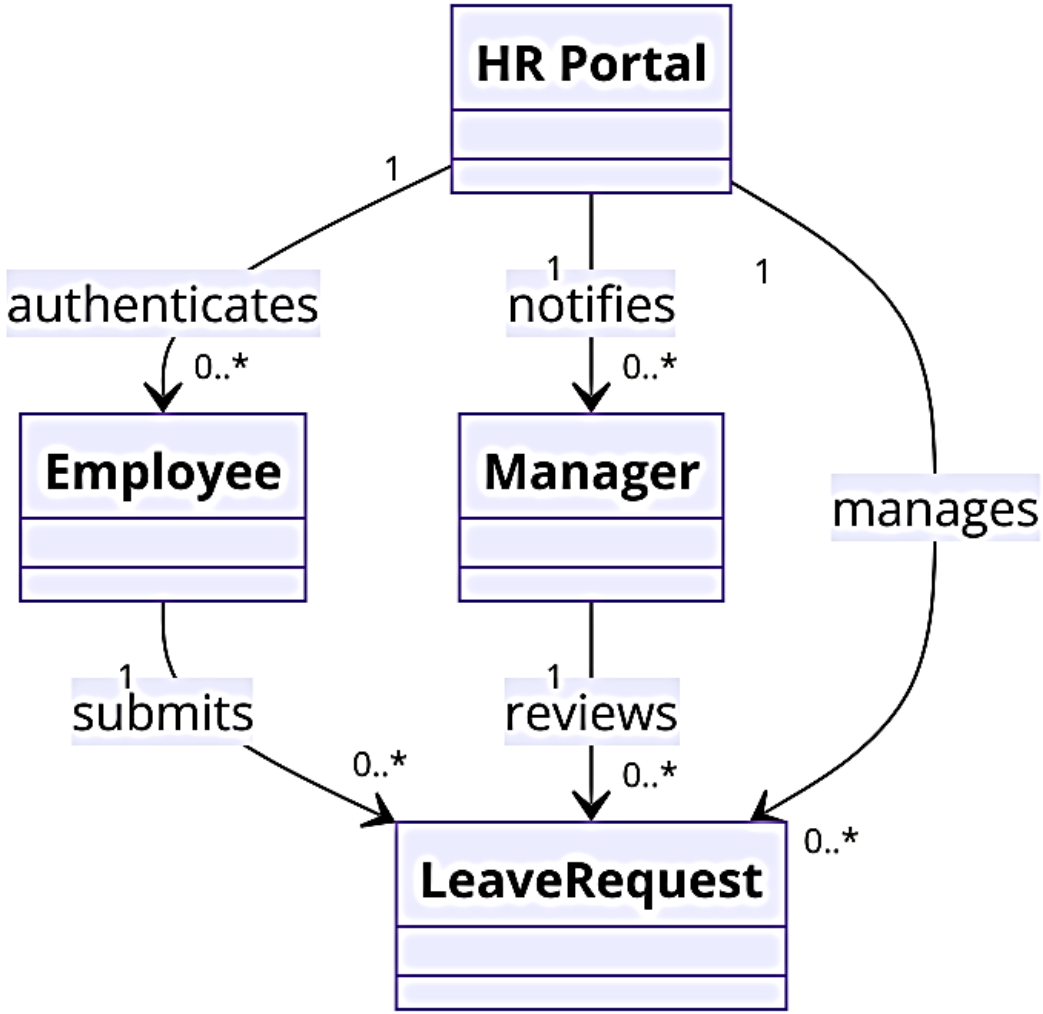
CLASS VISIBILITY MODIFIERS

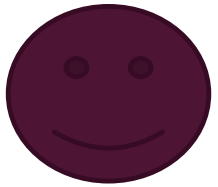


CLASS RELATIONSHIPS

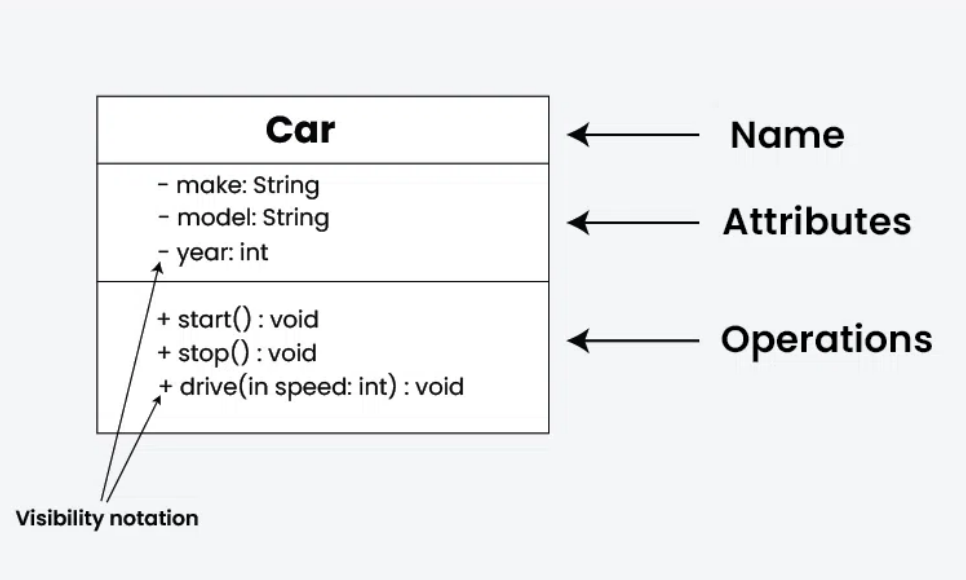


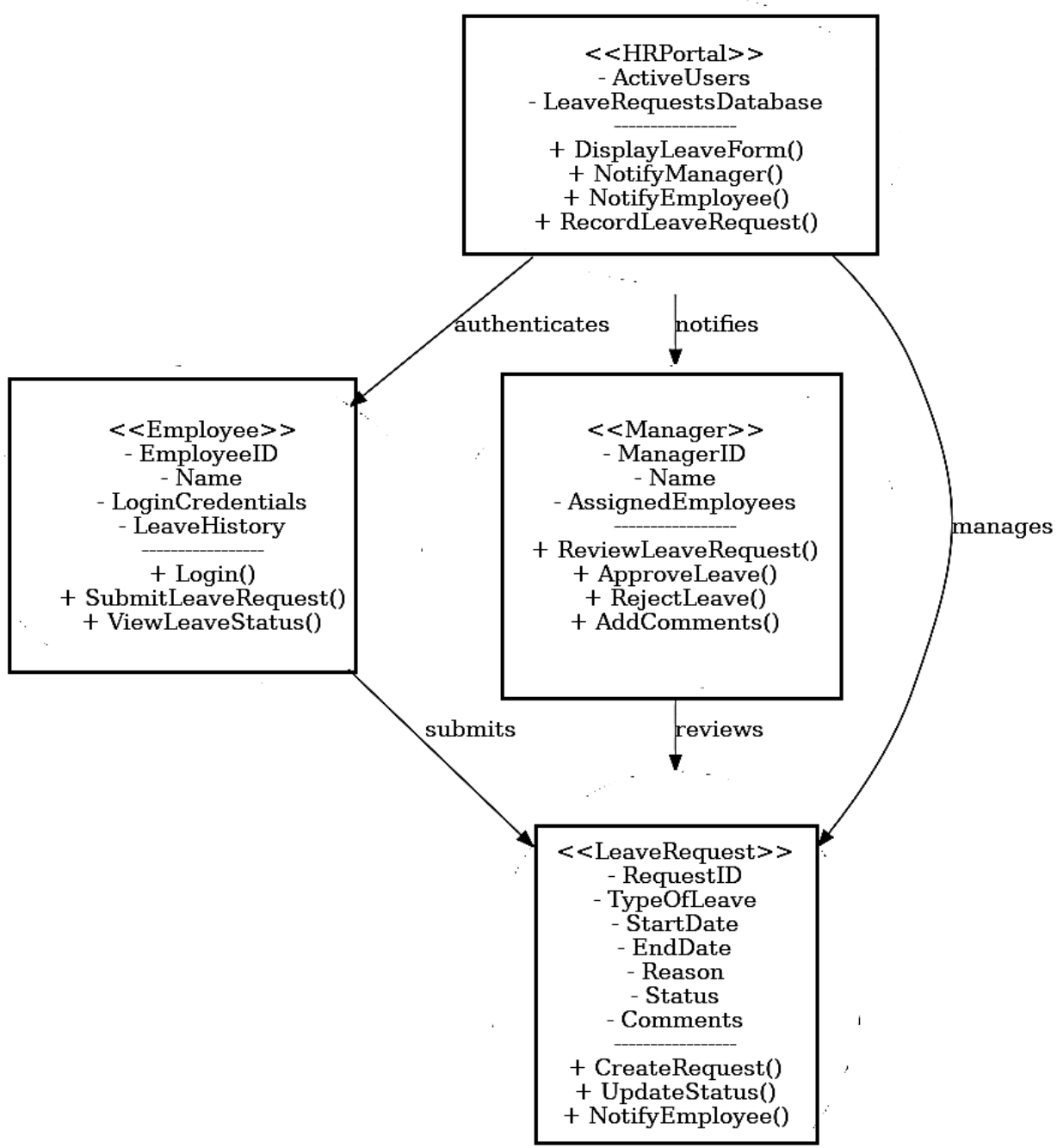
EXTRACTING CLASSES FROM A USE CASE



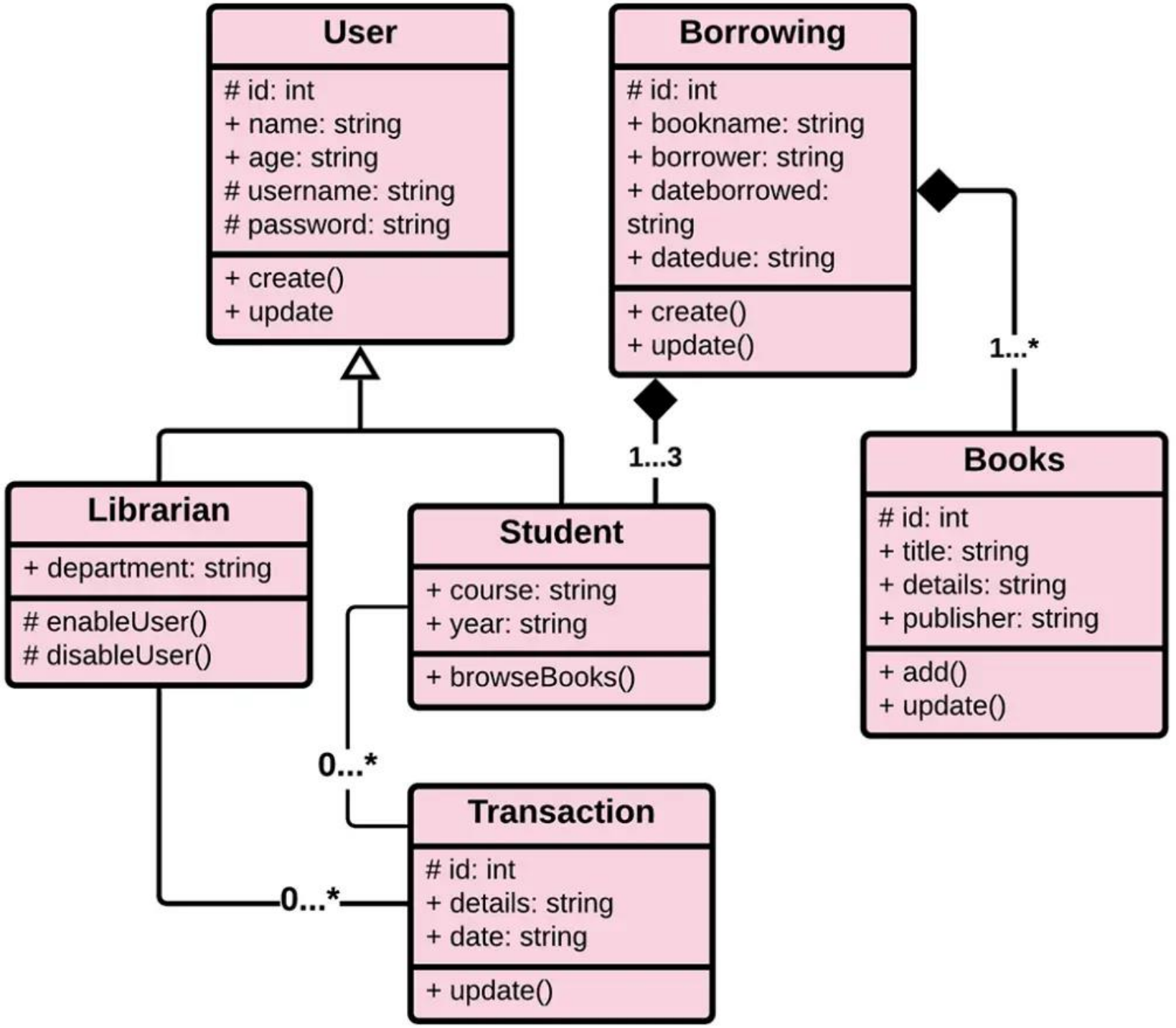


CLASS DIAGRAM





CLASS DIAGRAM



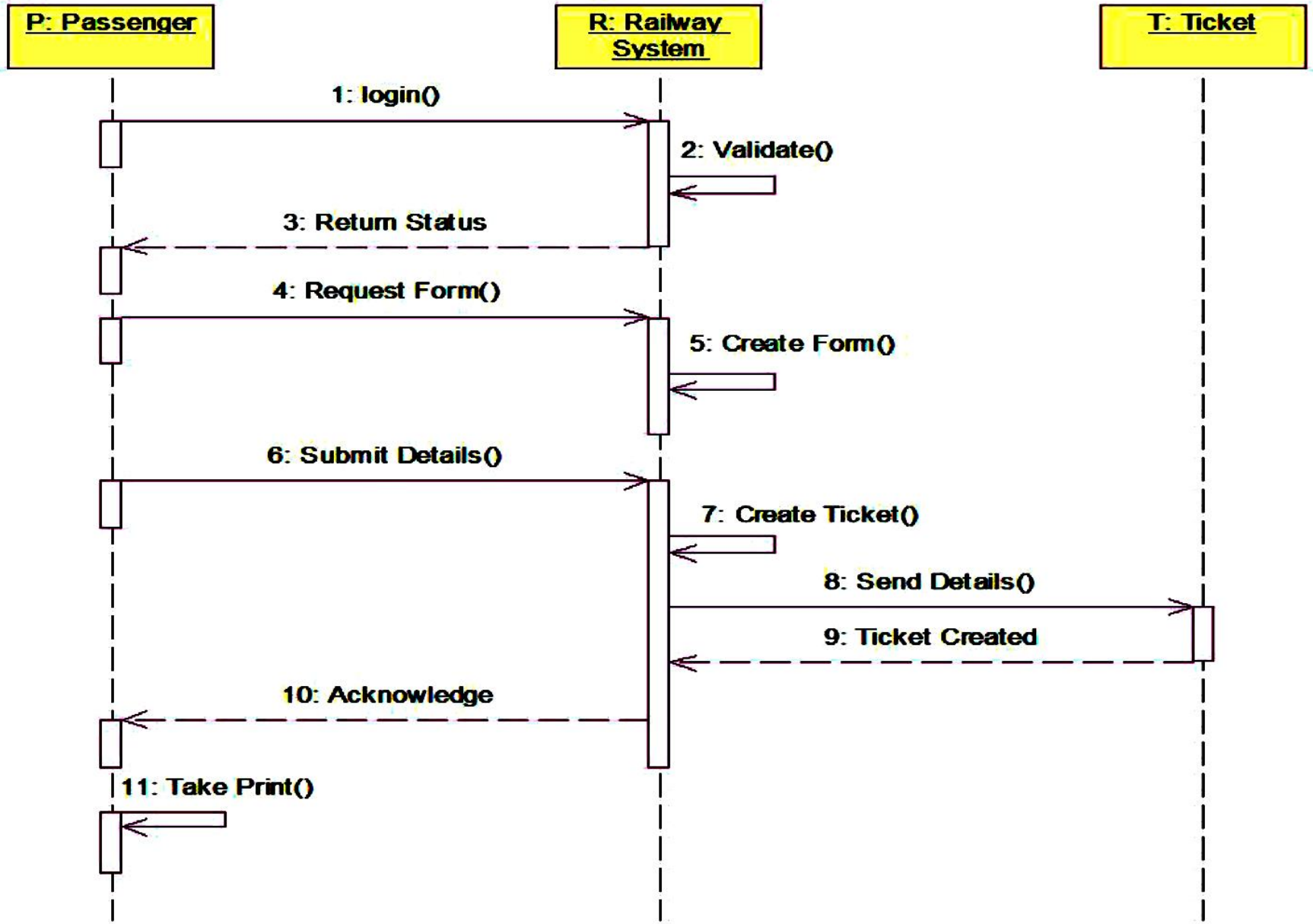
BRAIN TEASER 2

- Use the instructions provided and do as directed in the printed class assignment!!

SEQUENCE DIAGRAMS

- **Sequence diagrams** are interaction diagrams that illustrate the ordering of messages according to time.
- **Notations** − These diagrams are in the form of two-dimensional charts.
  - The objects that initiate the interaction are placed on the x–axis.
  - The messages that these objects send and receive are placed along the y–axis, in the order of increasing time from top to bottom.
- **Example** − A sequence diagram for the Railway reservation system

SEQUENCE DIAGRAMS



BENEFITS OF A SEQUENCE DIAGRAM

- It explores the real-time application.
- It depicts the message flow between the different objects.
- It has easy maintenance.
- It is easy to generate.
- It can easily update as per the new change in the system

ACTIVITY DIAGRAMS

- Activity diagrams are a type of behavioral diagram in UML (Unified Modeling Language) that visually represent workflows of stepwise activities and actions, including control flow and data flow.
- Activity diagrams are versatile and an essential part of UML, aiding developers, analysts, and stakeholders in understanding and optimizing workflows in software systems.

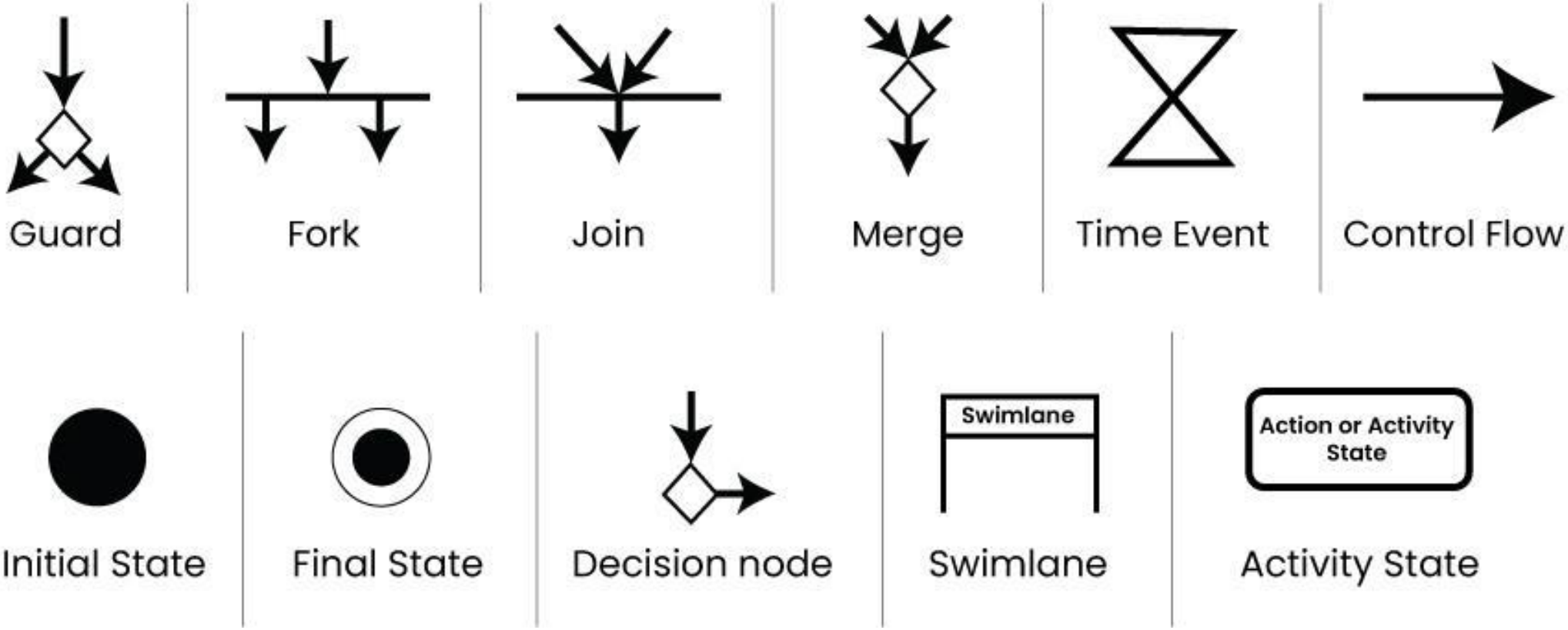
ACTIVITY DIAGRAMS - DEFINITION

- An activity diagram models the **workflow** or **business processes** in a system.
- It describes the **sequence of activities** and the flow of control between them.
- Useful for:
  - Modeling business processes
  - Understanding complex use cases
  - Workflow analysis

ACTIVITY DIAGRAM - ELEMENTS & NOTATION

- **Activities:** Represented by rounded rectangles.
- **Transitions:** Arrows indicating the flow between activities.
- **Decision Nodes:** Represented by diamonds for branching.
- **Fork/Join Nodes:** Indicate parallel processing.
- **Swimlanes:** Represent different actors or components involved.

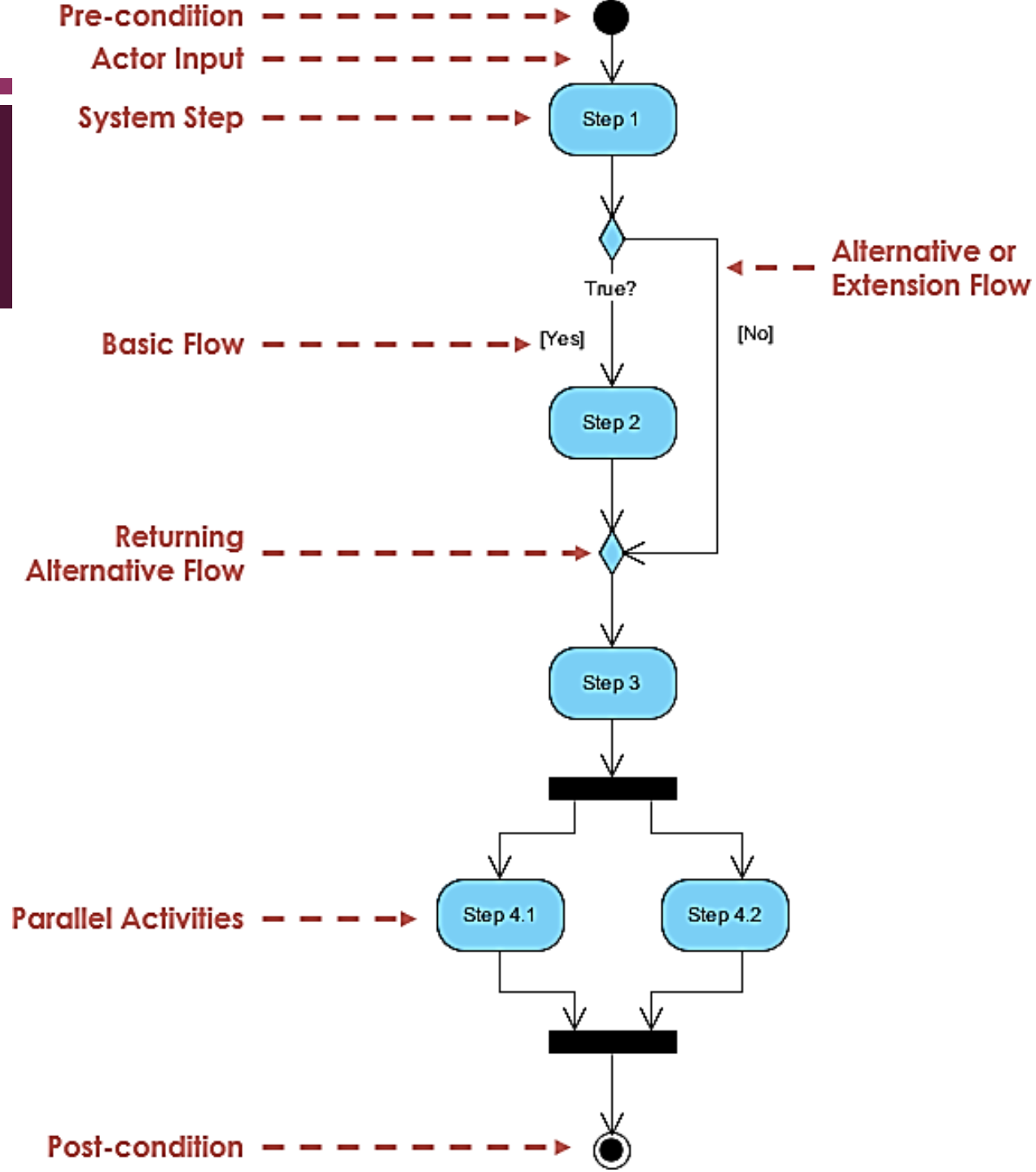
ACTIVITY DIAGRAM NOTATIONS



WHEN TO USE?

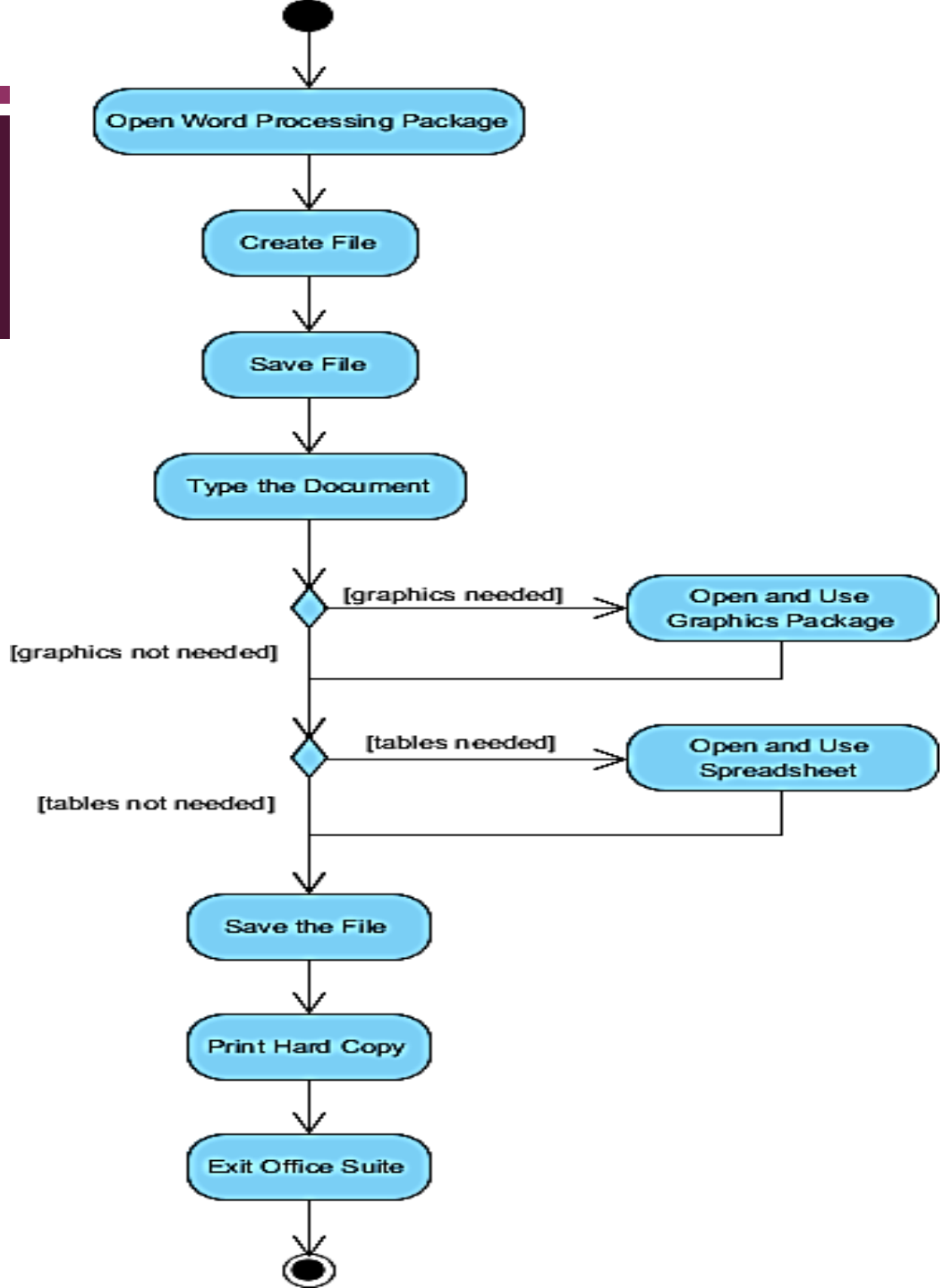
- Guard: Represents a condition that must be true for a transition to occur.
- Fork: Splits a single control flow into multiple concurrent flows.
- Join: Combines multiple concurrent flows into a single control flow.
- Merge: Combines alternative flows into one path.
- Time Event: Represents an event triggered by a specific time condition.
- Control Flow: Shows the flow of control between activities.
- Initial State: Indicates the starting point of the activity.
- Final State: Represents the termination point of the activity.
- Decision Node: Represents a choice between alternative flows based on conditions.
- Swimlane: Organizes activities into different roles or responsibilities.
- Activity State: Represents a specific action or process in the activity.

ACTIVITY DIAGRAM



ACTIVITY DIAGRAM - MODELING A WORD PROCESSOR

- **The workflow for a word process to create a document through the following steps:**
- Open the word processing package.
- Create a file.
- Save the file under a unique name within its directory.
- Type the document.
- If graphics are necessary, open the graphics package, create the graphics, and paste the graphics into the document.
- If a spreadsheet is necessary, open the spreadsheet package, create the spreadsheet, and paste the spreadsheet into the document.
- Save the file.
- Print a hard copy of the document.
- Exit the word processing package.

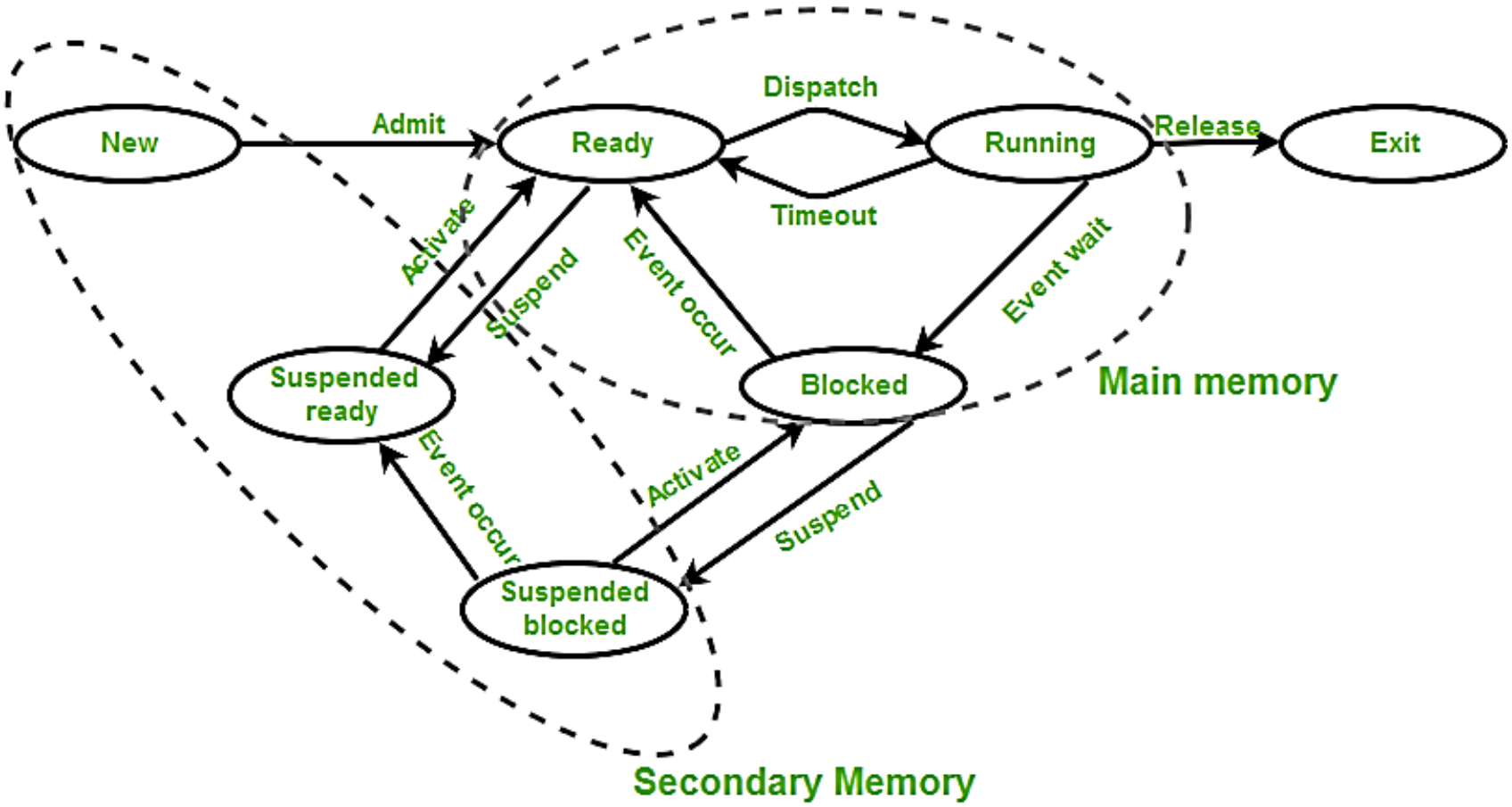


STATE MACHINE DIAGRAMS - DEFINITION

- A state machine diagram models the **states** an object can be in and the **transitions** between those states.
- It focuses on how the **state of an object changes** in response to events.
- Useful for:
  - Modeling object lifecycles
  - Understanding system behavior in response to events

Remember operating system process states.

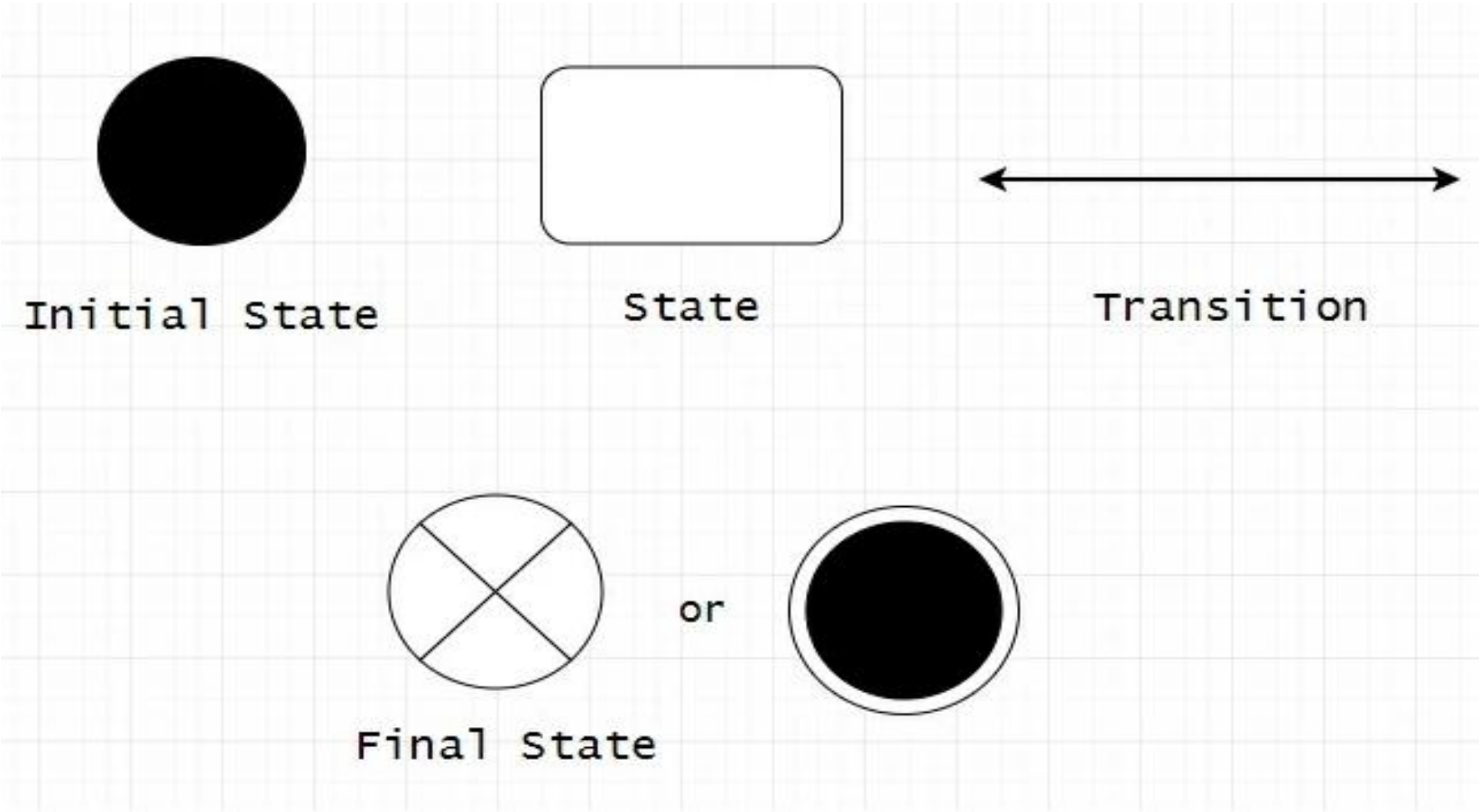
STATES OF AN OPERATING SYSTEM



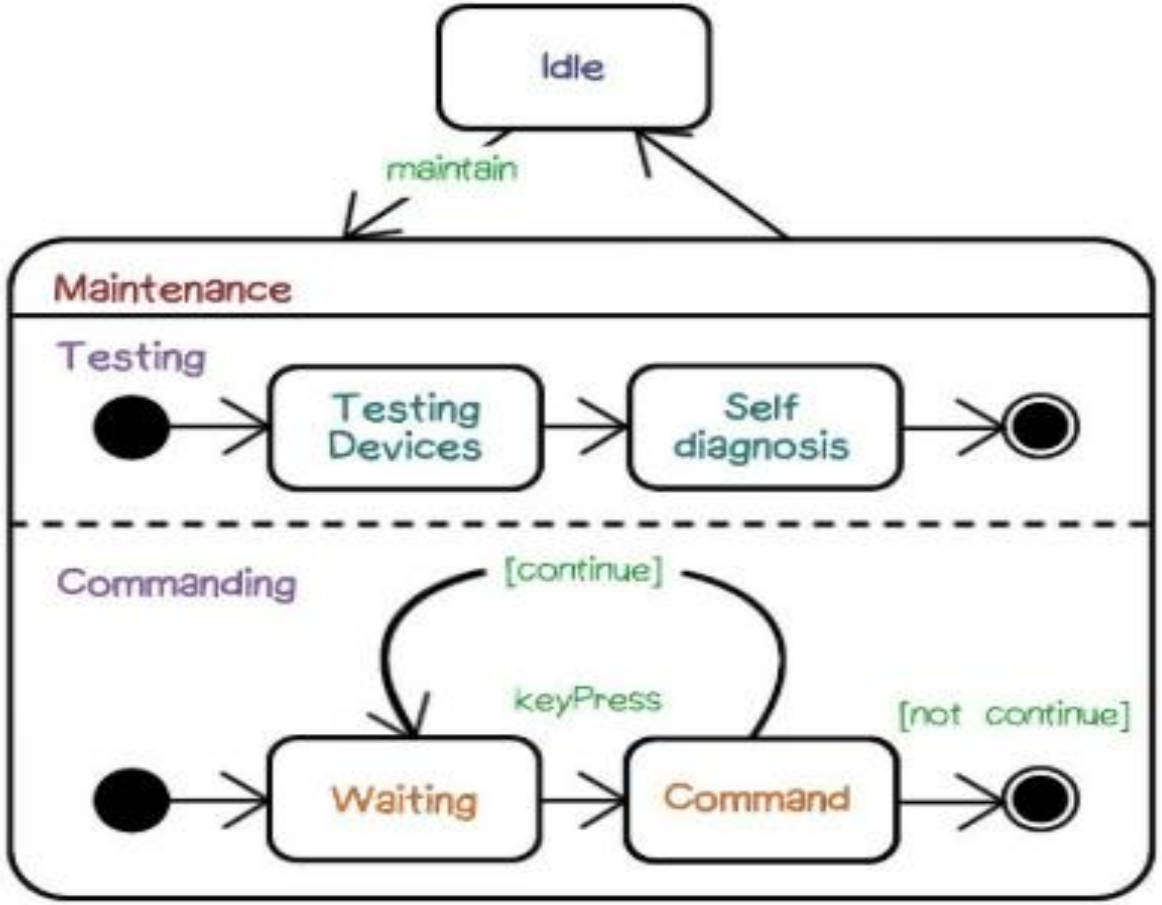
STATE MACHINE DIAGRAM - ELEMENTS & NOTATION

- **States:** Represented by rounded rectangles with the state name inside.
- **Transitions:** Arrows indicating movement between states.
- **Initial State:** Represented by a filled black circle.
- **Final State:** Represented by a black circle with a white center.
- **Events & Actions:** Events trigger transitions; actions may be executed during transitions.

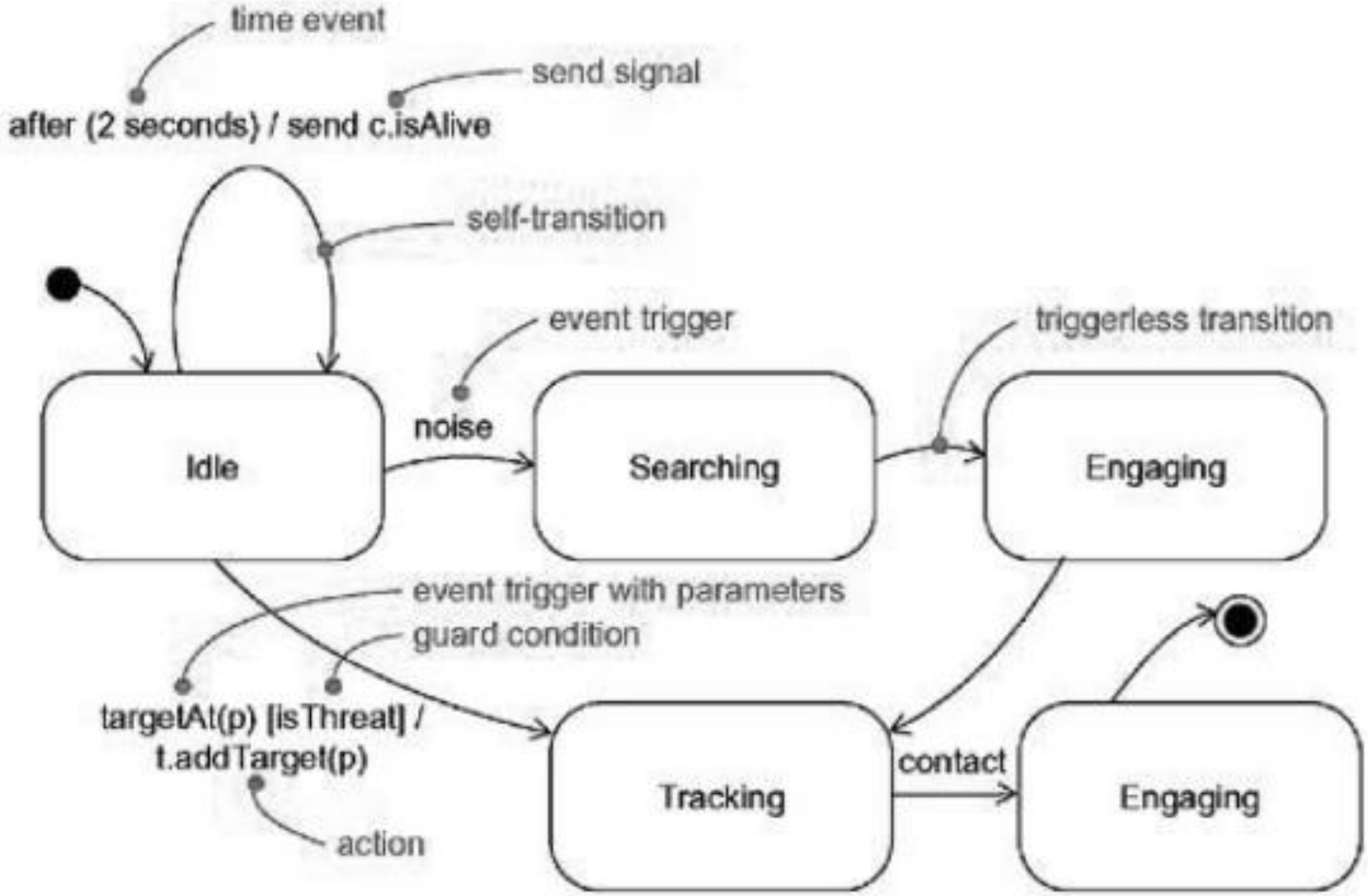
NOTATIONS



EXAMPLE


Idle
maintain
Maintenance
Testing
Testing Devices
Self diagnosis
Commanding
[continue]
keyPress
[not continue]
Waiting
Command

EXAMPLE 2


time event
send signal
after (2 seconds) / send c.isAlive
self-transition
event trigger
triggerless transition
noise
Idle
Searching
Engaging
event trigger with parameters
guard condition
targetAt(p) [isThreat] /
t.addTarget(p)
action
Tracking
contact
Engaging

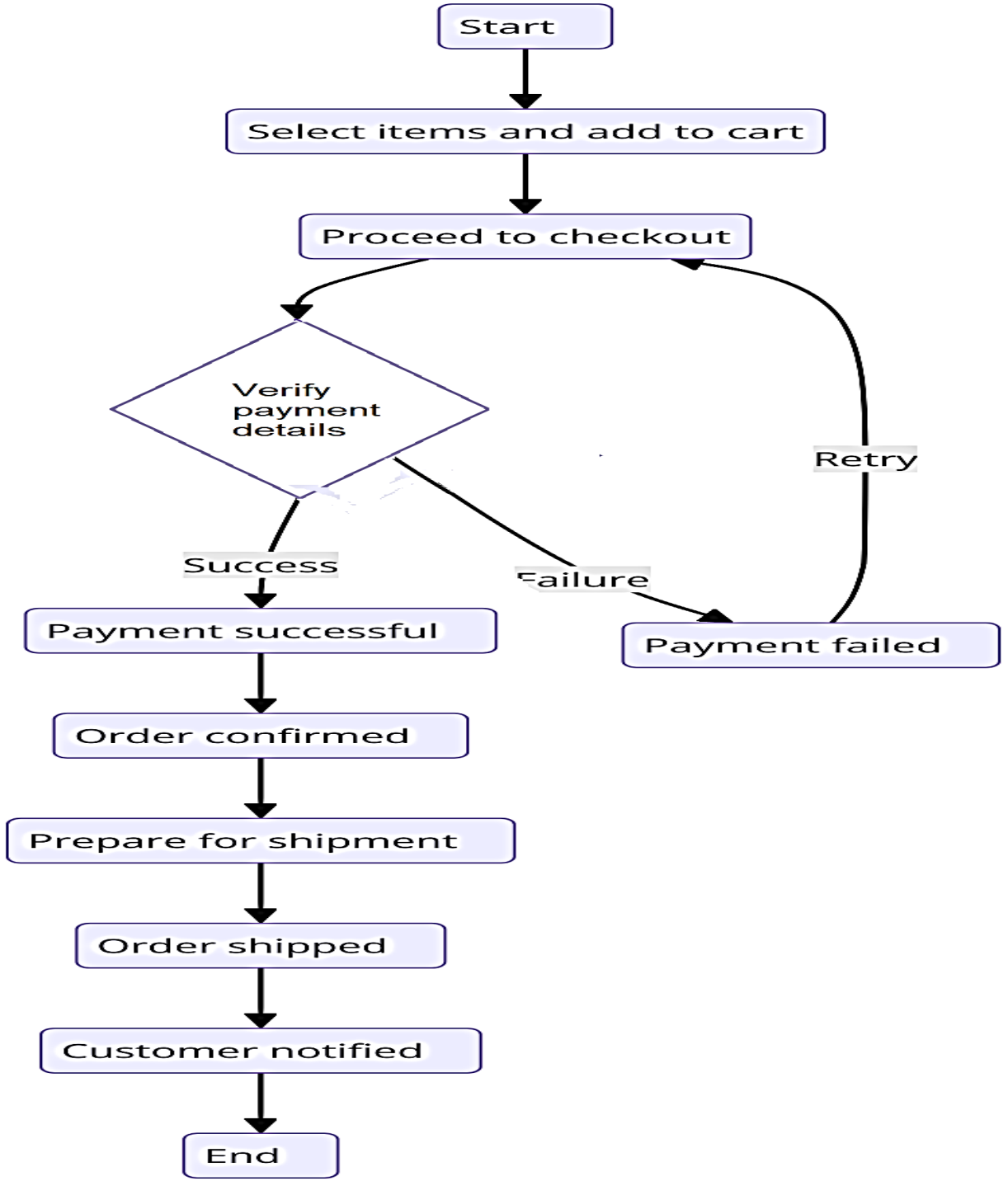
COMPARISON

Aspect	Activity Diagram	State Machine Diagram
Focus	Workflow of activities	States and transitions of an object
Purpose	Process modeling	Behavior modeling
Representation	Activities, decisions, swimlanes	States, events, transitions
Usage	Business process, use case modeling	Object lifecycle, system behavior

BRAIN TEASER: ACTIVITY DIAGRAM

- **Scenario 1:**
  A customer places an order on an e-commerce website. The order process involves the following steps:
- The customer selects items and adds them to the cart.
- The customer proceeds to checkout.
- The system verifies payment details.
- If payment is successful, the order is confirmed; otherwise, the customer is notified to retry payment.
- The order is prepared for shipment.
- The order is shipped, and the customer is notified.
- **Task:**
  Draw an **activity diagram** representing this process.

SOLUTION


Start
Select items and add to cart
Proceed to checkout
Verify payment details
Success
Failure
Retry
Payment successful
Payment failed
Order confirmed
Prepare for shipment
Order shipped
Customer notified
End

SCENARIO 2: STATE TRANSITION DIAGRAM

- **Scenario:**
  Consider a **turnstile** system at the entrance of a metro station. The turnstile has two states:
- **Locked** – It is locked initially and only unlocks when a valid ticket is inserted.
- **Unlocked** – When the turnstile is unlocked, the passenger can pass through. Once the passenger passes, it goes back to the locked state.
- If an invalid ticket is inserted, the turnstile remains locked.
- **Task:**
  Develop a **state transition diagram** for the turnstile system showing the states and transitions triggered by events (e.g., "insert valid ticket," "passenger passes").

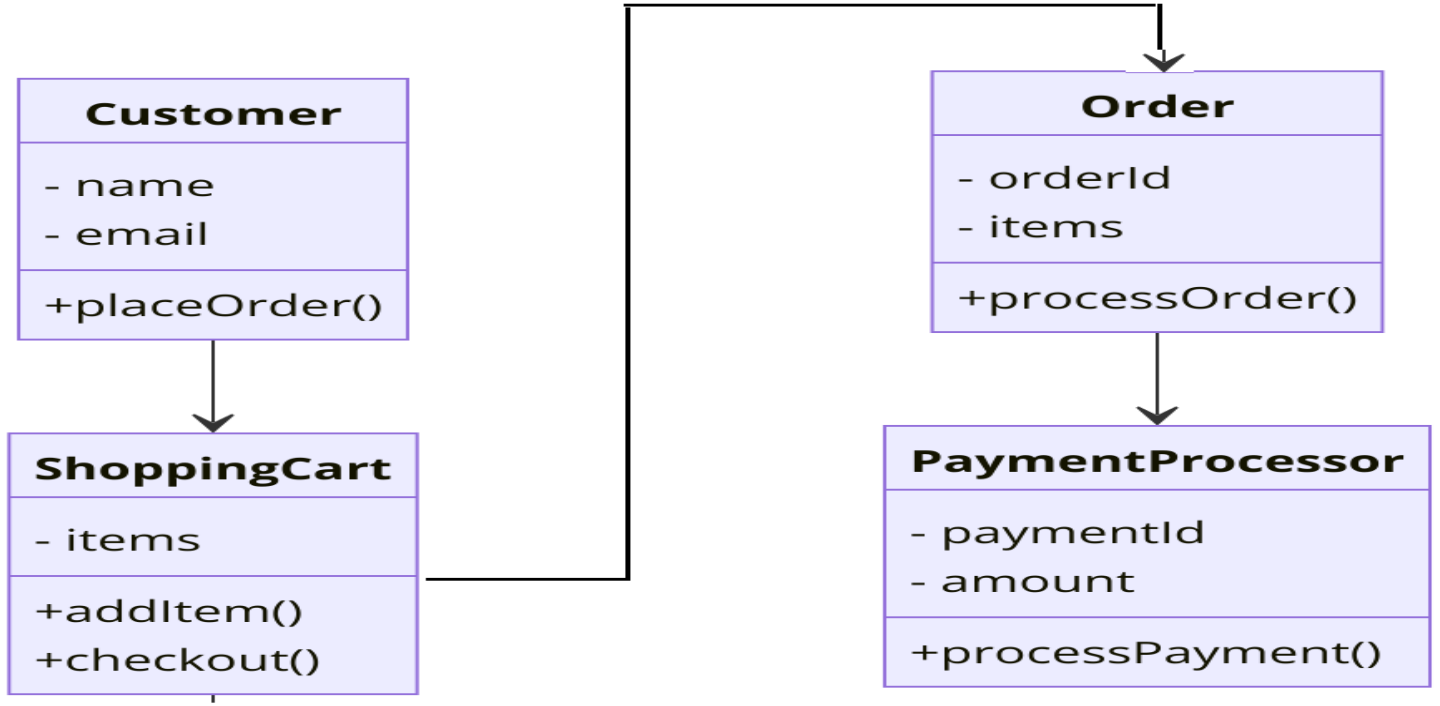
MODELING INTERACTIONS AND OBJECT LIFECYCLES

- Modeling interactions and object lifecycles **involves capturing** how **objects in a system interact** with each other **during the execution of specific tasks** (interactions) and tracking the states an object goes through from creation to destruction (lifecycles).
- This is a critical part of **object-oriented design** to ensure the system behaves as expected under different scenarios.
- **Mapping Use Cases to Objects**
- Explanation**:**
  - Use cases describe user interactions with the system. The next step is to identify the objects responsible for implementing the use case.
- Steps:
  - Analyze the **use case** text to find nouns (potential objects) and verbs (methods/behaviors).
  - Map these to **classes and methods**.
  - Use **sequence diagrams** to detail object interactions for a specific use case.

MAPPING USE CASES TO OBJECTS

- Example: Use Case "A customer places an order online by adding items to their shopping cart, checking out, and paying."
- Identified Objects:
  - Customer
  - ShoppingCart
  - Order
  - PaymentProcessor
- Responsibilities:
  - Customer: Initiates actions.
  - ShoppingCart: Stores items.
  - Order: Represents the order.
  - PaymentProcessor: Handles payment.

EXTRACTING CLASSES



MODELING ASSOCIATIONS AND AGGREGATIONS

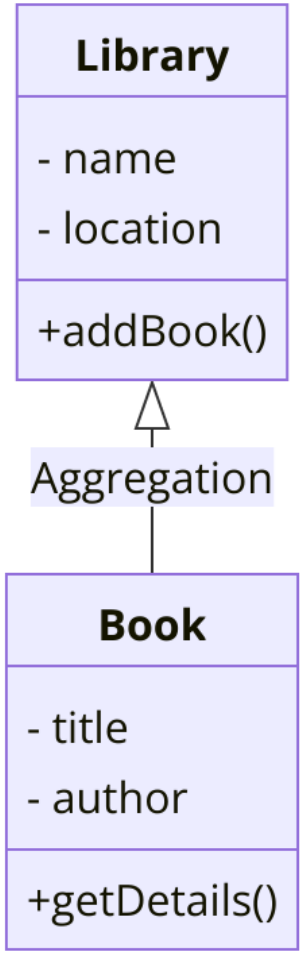
- Explanation:
  - Association: A basic relationship between two objects (e.g., "A customer places an order").
  - Aggregation: A "whole-part" relationship where the part can exist independently of the whole.
- Example:
- Class Diagram for Aggregation: Customer → Order

AGGREGATION

Relationship:

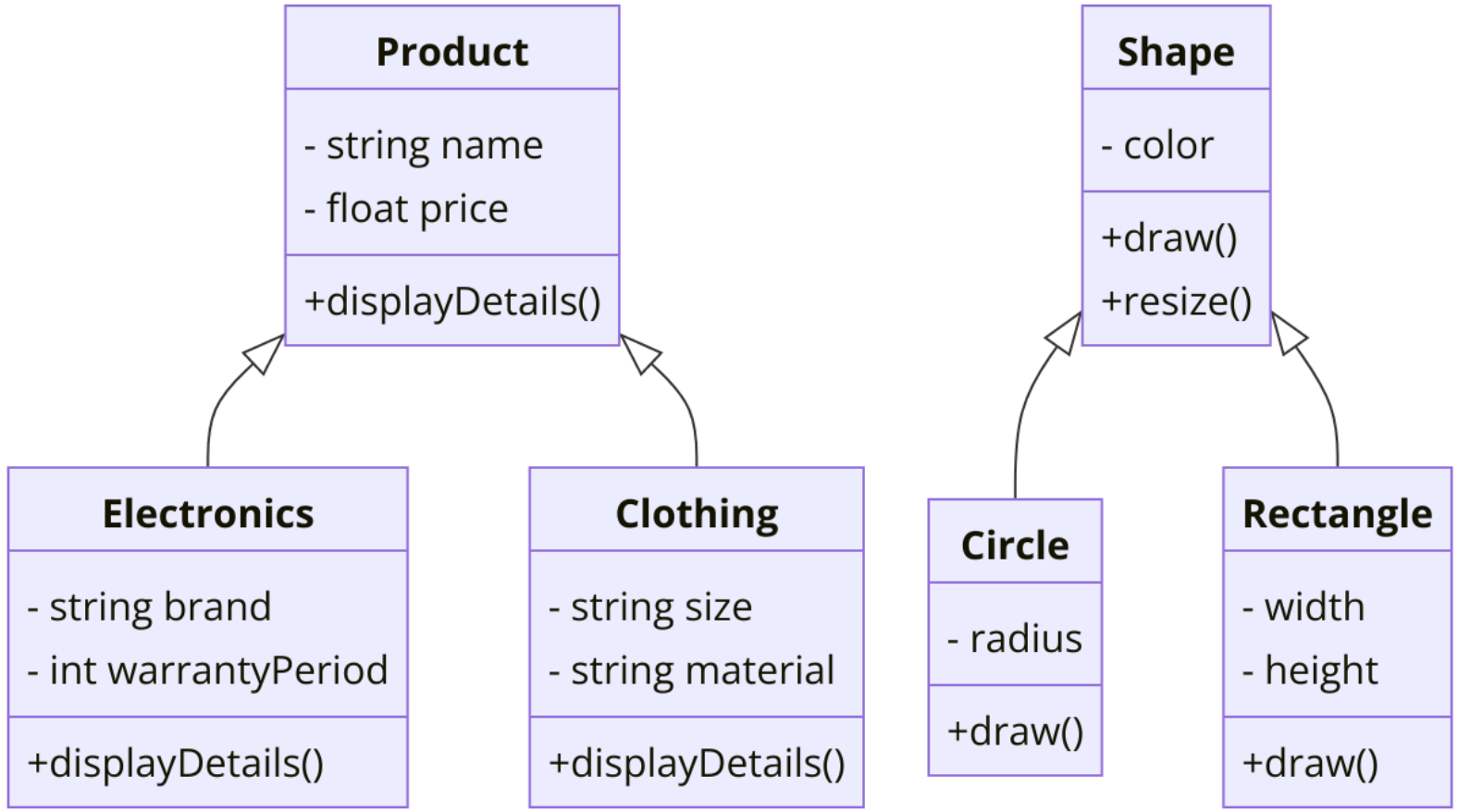
•The <> symbol represents an **aggregation** relationship:

- •A **Library** contains **Books**, but books can exist independently of the library.
- •This is different from composition, where the contained object (e.g., Book) would not exist without the parent object (e.g., Library).



MODELING INHERITANCE RELATIONSHIPS

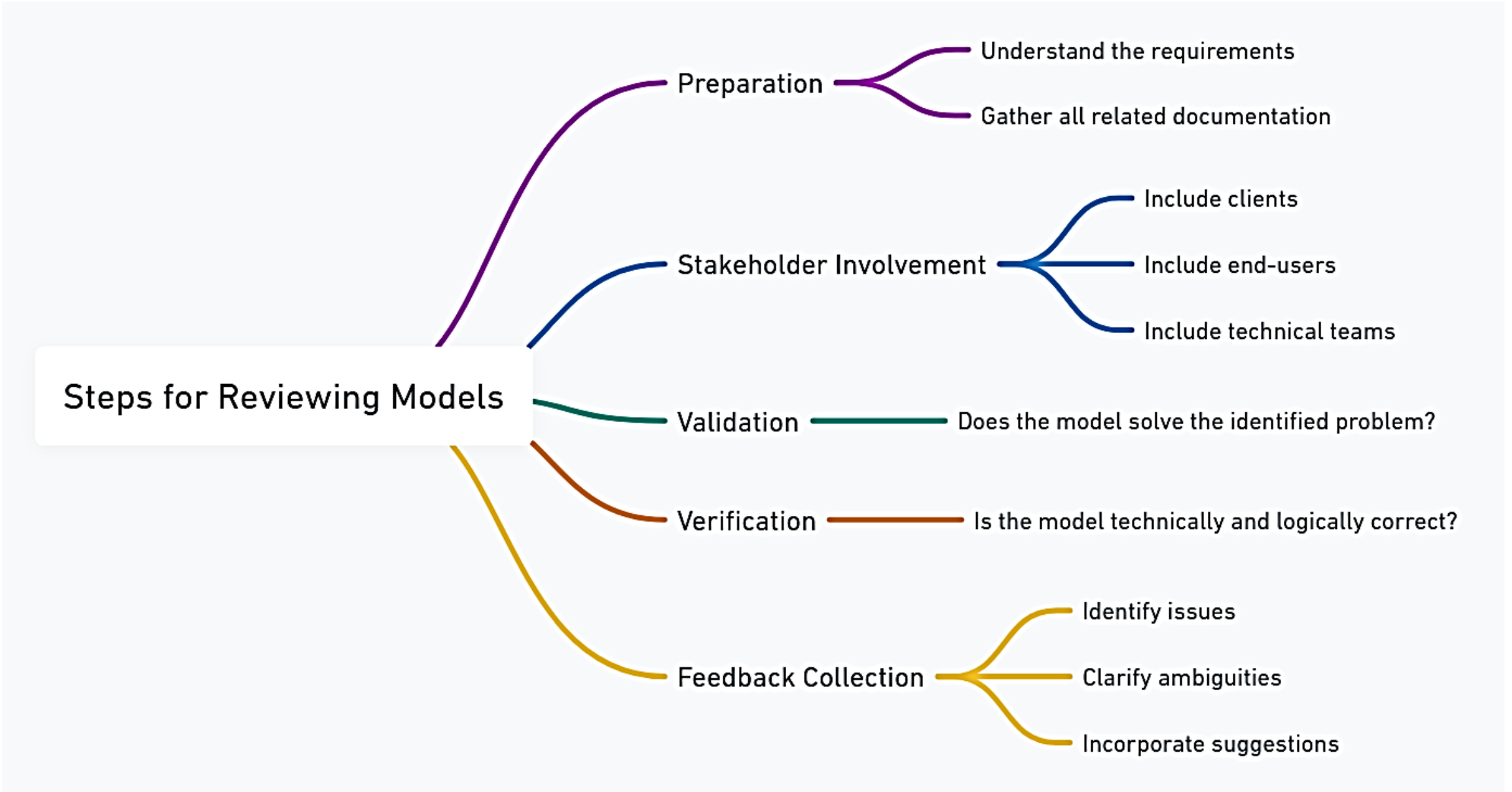
- Inheritance enables reuse and creates a hierarchy where subclasses inherit from a superclass.
- Subclasses can also override methods.



REVIEWING ANALYSIS AND DESIGN MODELS

- **What is it?**
  - A process of validating and verifying models (e.g., ERD, UML diagrams, process flows) to ensure accuracy and alignment with requirements.
- **Why is it important?**
  - Identify gaps or errors early.
  - Ensure models align with client expectations.
  - Reduce rework and costly changes later.
- **Key Components to Review:**
  - Completeness
  - Consistency
  - Clarity
  - Traceability to requirements

STEPS FOR REVIEWING MODELS



CLIENT SIGN-OFF PROCEDURES

- **What is Client Sign-Off?**
  - A formal agreement from the client, confirming that the deliverables meet their expectations and requirements.
- **Why is it Necessary?**
  - Establishes a clear approval point.
  - Reduces misunderstandings.
  - Ensures accountability.

STEPS FOR CLIENT SIGN-OFF

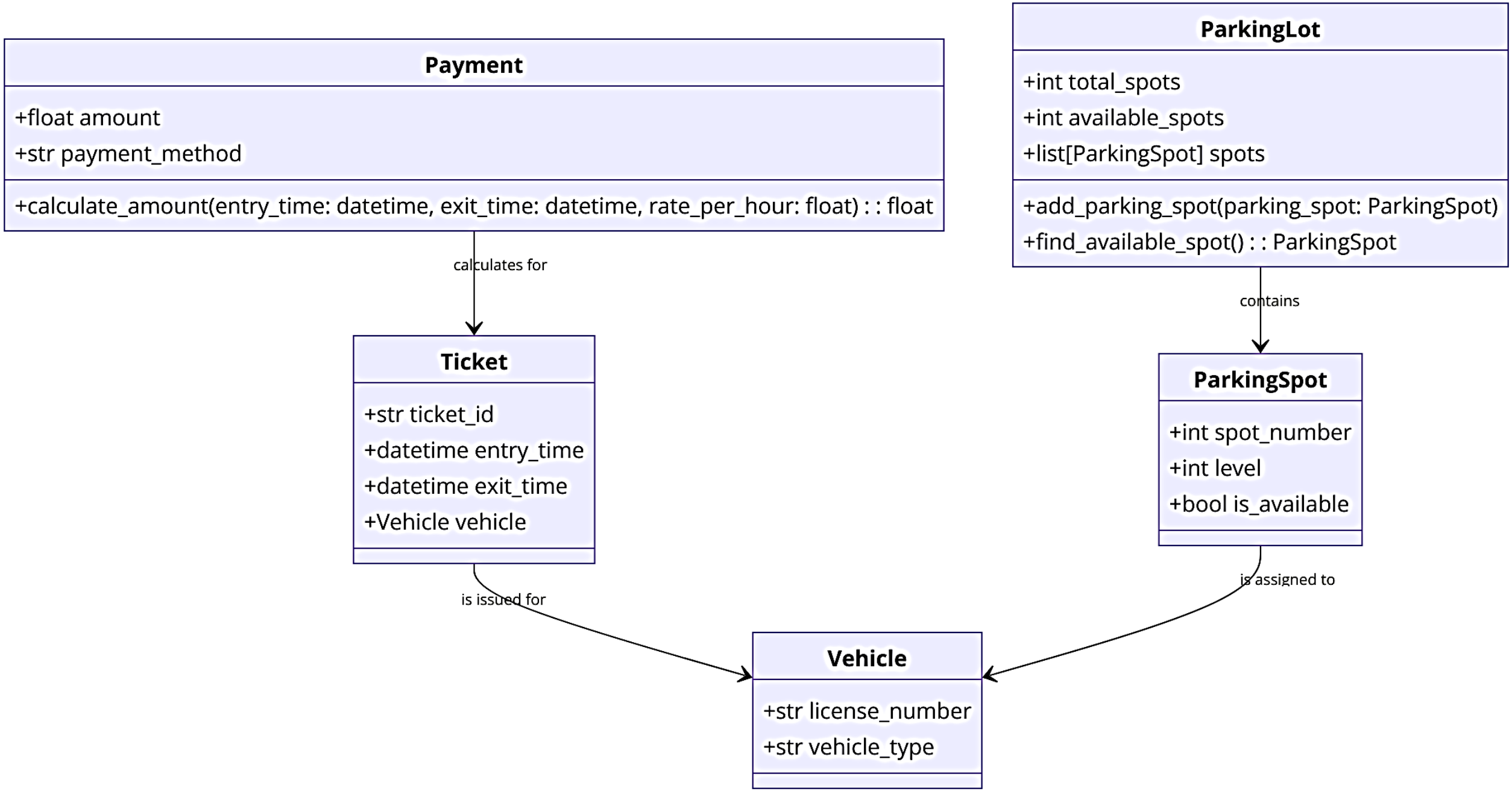
- **Prepare Deliverables**:
  - Ensure completeness, clarity, and documentation of work.
- **Presentation**:
  - Demonstrate deliverables to the client.
- **Feedback and Revisions**:
  - Address concerns and refine deliverables if needed.
- **Approval**:
  - Obtain formal sign-off through documents or digital systems.

BEST PRACTICES FOR CLIENT SIGN-OFF

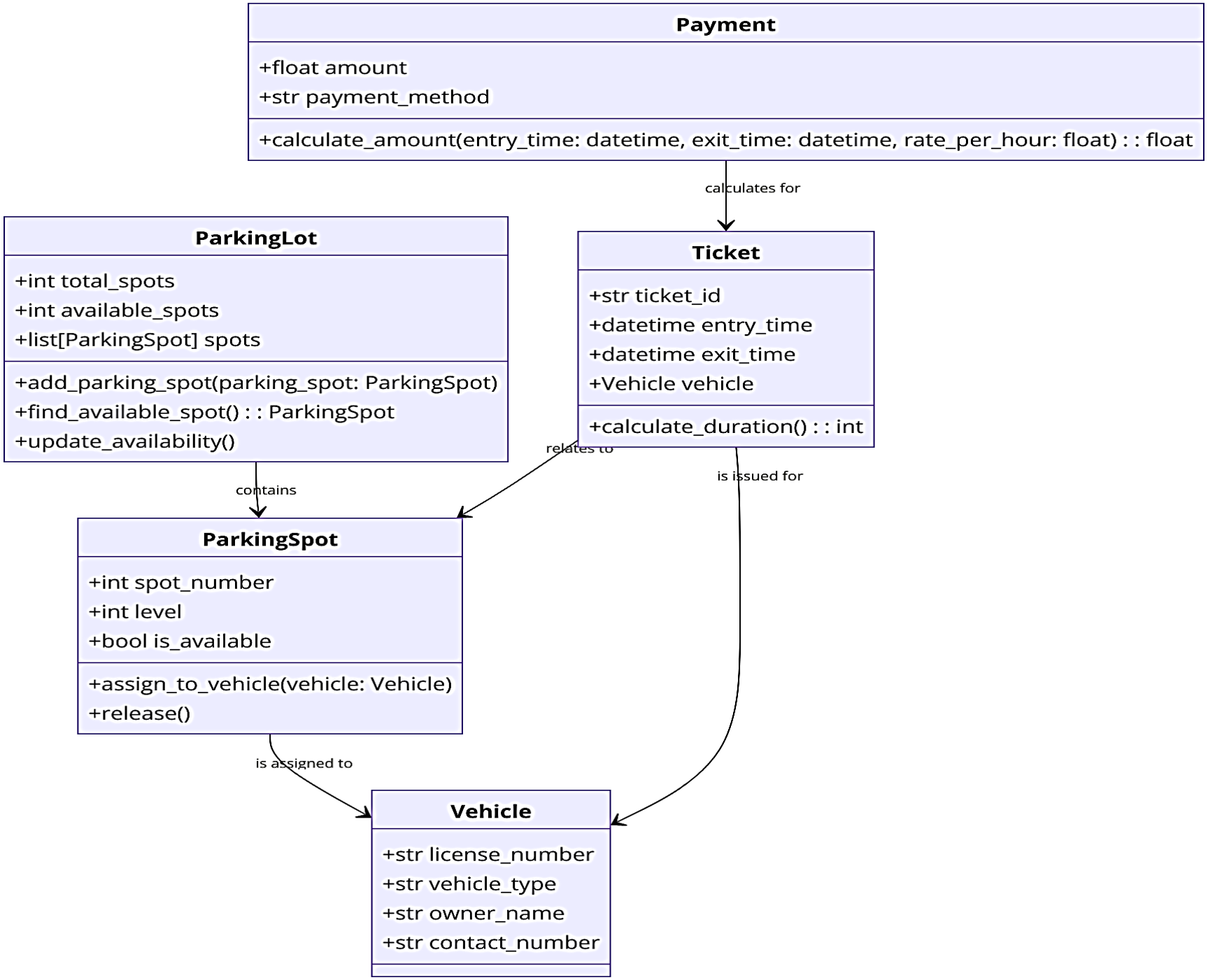
- Involve clients throughout the project lifecycle.
- Maintain clear and open communication.
- Provide detailed documentation of deliverables.
- Use structured templates for sign-off.
- Keep records for future reference.

BRAIN TEASER

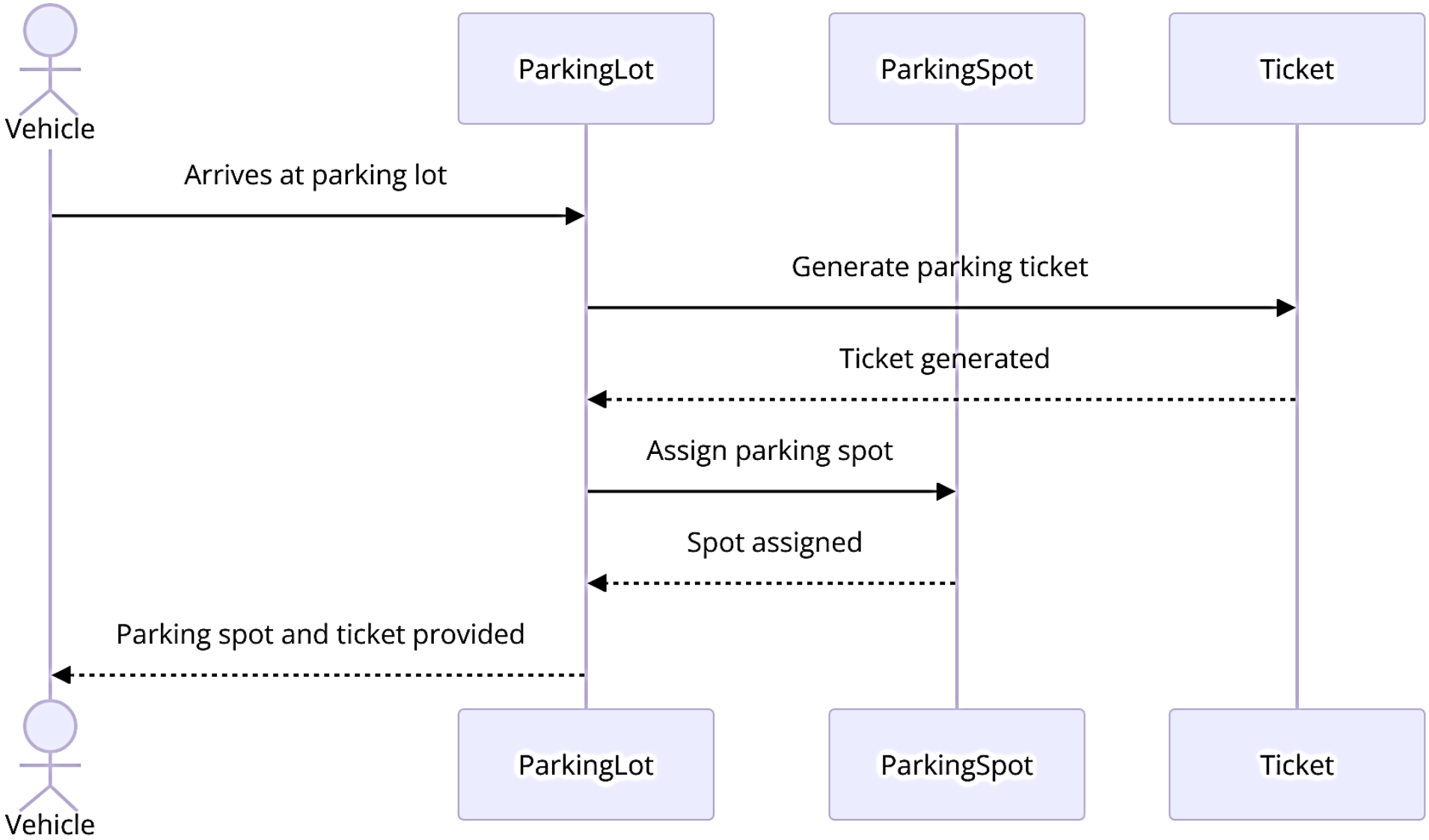
- Solve the question provided in Cheat Sheet 4


Payment
+float amount
+str payment_method
+calculate_amount(entry_time: datetime, exit_time: datetime, rate_per_hour: float) : : float
ParkingLot
+int total_spots
+int available_spots
+list[ParkingSpot] spots
+add_parking_spot(parking_spot: ParkingSpot)
+find_available_spot() : : ParkingSpot
calculates for
contains
Ticket
+str ticket_id
+datetime entry_time
+datetime exit_time
+Vehicle vehicle
ParkingSpot
+int spot_number
+int level
+bool is_available
is issued for
is assigned to
Vehicle
+str license_number
+str vehicle_type

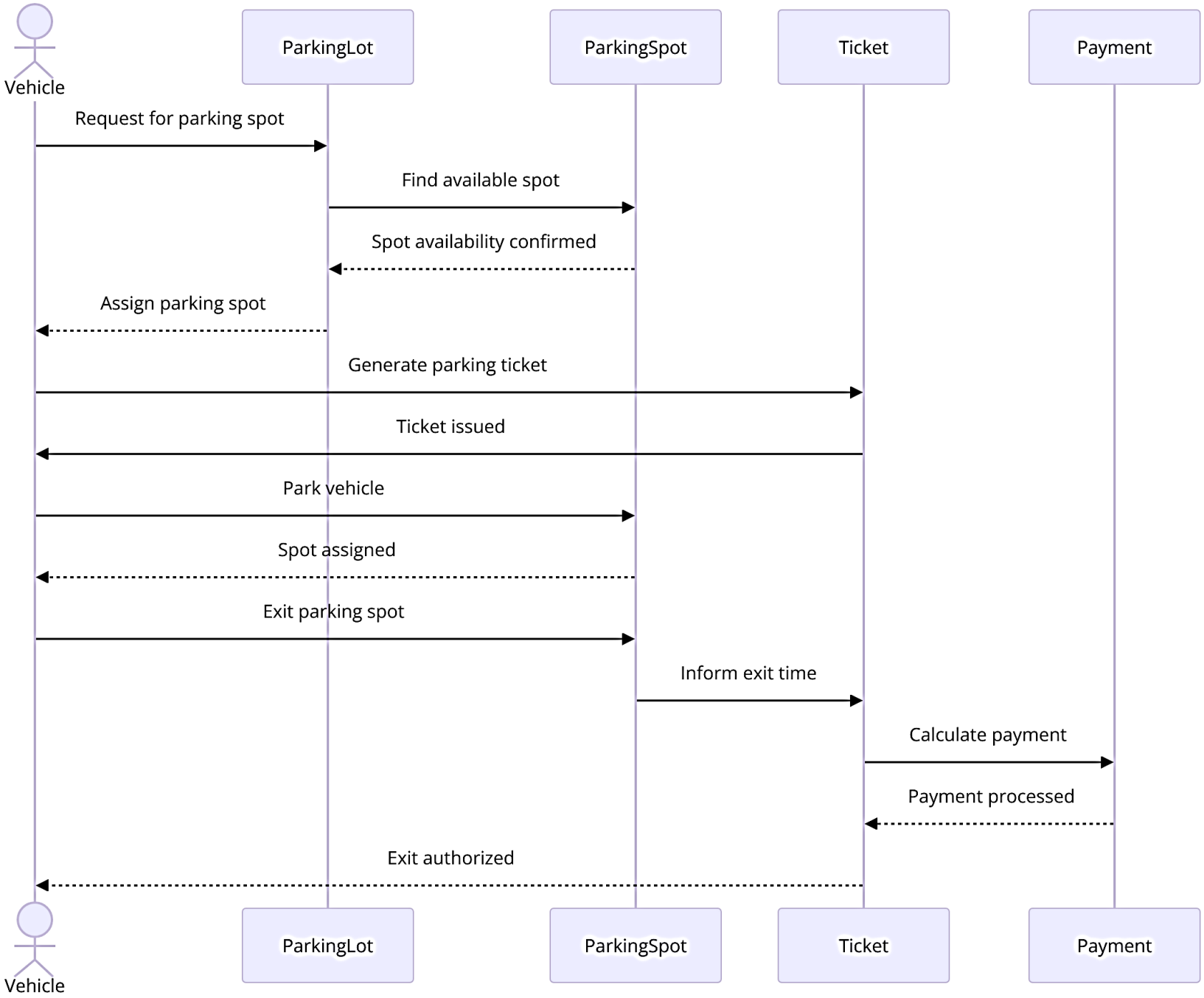
EXTENDED DESIGN



MODEL THE INTERACTION FOR A VEHICLE ENTERING THE PARKING LOT:



SEQUENCE DIAGRAM: COMPLETE SYSTEM



THANK YOU

Any Questions??